



Arab Academy for Science, Technology, and Maritime Transport
College of
Computing and Information Technology
Alexandria

Smart Vision Assist

A Thesis submitted in partial fulfillment of the requirements of
Bachelor Degree of

by

Aya Abdelhakam

Software Engineering

Khadija Mohamed

Software Engineering

Hana Khamis

Software Engineering

Habiba Aboraya

Computer Science

College of Computing and Information Technology, AASTMT, 2026

Supervised By

Prof. DR .Walid Rabie

DR .Yasmine Nagy

Alexandria, 2026

Abstract

Smart Vision Assist introduces the development and evaluation of an intelligent mobile application designed to assist visually impaired individuals during indoor navigation. The application empowers users to perceive their surroundings more safely by detecting obstacles, estimating their relative distances, and providing immediate multimodal feedback through voice and vibration notifications. By combining Computer Vision and Deep Learning technologies, Smart Vision Assist aims to bridge the gap between environmental perception and independent mobility.

This research emphasizes the application's ability to deliver an affordable, portable, and user-centered assistive solution that enhances environmental awareness without requiring specialized hardware. The implementation process focused on modular system architecture, real-time Artificial Intelligence processing, and seamless interaction between mobile and backend components. Through testing and evaluation, Smart Vision Assist was assessed for its functionality, performance, and usability in indoor environments. The findings of this work contribute to the advancement of assistive technologies by offering an accessible and intelligent approach that promotes safer navigation and greater independence for visually impaired individuals.

Acknowledgment

We would like to extend our deepest gratitude to everyone who contributed to the success of this project. Firstly, we would like to thank our supervisors, **Prof. Dr. Walid Rabie** and **Dr Yasmine Nagy** for their invaluable guidance, insightful feedback, and continuous support throughout the project.

We are also immensely grateful to the faculty members of the Arab Academy for Science, Technology and Maritime Transport, particularly the College of Computing and Information Technology, for their academic supervision, mentorship, and dedication to fostering an environment of intellectual growth and innovation. Their commitment to academic and professional excellence has been a constant source of inspiration for our team.

Furthermore, we would like to express our appreciation to all individuals and professionals who provided feedback and insights regarding career development challenges, which greatly contributed to refining the objectives and vision of our platform.

Lastly, we would like to extend our heartfelt thanks to our families and friends for their unwavering support, patience, and encouragement throughout this journey. Their belief in our abilities, and their constant motivation, have been a source of strength during both the challenges and successes of this project. To everyone mentioned above, and to all those who have contributed directly or indirectly, we offer our deepest and most sincere thanks. Your support has been invaluable in making this thesis and the development of the Vision app a reality.

Table of Contents

Abstract.....	II
Acknowledgment.....	III
List of Figures.....	IX
List of Tables.....	X
Abbreviations	XI
Chapter 1 - Introduction	1
1.1 Motivation.....	1
1.2 Problem Statement	2
1.3 Objectives.....	2
1.4 Problem Complexity	3
1.5 Constraints	4
1.6 Standards	5
1.7 Feasibility Study and Business Canvas	6
1.8 Thesis Organization.....	11
Chapter 2 – Background	19
2.1 Introduction.....	19
2.2 Computer Vision.....	20
2.2.1 Definition and Overview	20
2.2.2 Key Features.....	20
2.2.3 Application in the Project.....	20
2.2.4 Advantages and Disadvantages	20
2.3 Deep Learning	21
2.3.1 Definition and Overview	21
2.3.2 Key Features.....	21
2.3.3 Application in the Project.....	22
2.3.4 Advantages and Disadvantages	22
2.4 Convolutional Neural Networks (CNNs).....	22
2.4.1 Definition and Overview	22
2.4.2 Key Features.....	23
2.4.3 Application in the Project.....	23
2.4.4 Advantages and Disadvantages	23
2.5 Object Detection	24
2.5.1 Definition and Overview	24
2.5.2 Key Features.....	24
2.5.3 Application in the Project.....	24
2.5.4 Advantages and Disadvantages	24
2.6 You Only Look Once (YOLO11)	25
2.6.1 Definition and Overview	25
2.6.2 Key Features.....	25

2.6.3 Application in the Project.....	26
2.6.4 Advantages and Disadvantages	26
2.7 <i>Monocular Depth Estimation</i>	27
2.7.1 Definition and Overview	27
2.7.2 Key Features.....	27
2.7.3 Application in the Project.....	27
2.7.4 Advantages and Disadvantages	28
2.8 <i>Monocular Depth Estimation at Scale (MiDaS)</i>	28
2.8.1 Definition and Overview	28
2.8.2 Key Features.....	29
2.8.3 Application in the Project.....	29
2.8.4 Advantages and Disadvantages	29
2.9 <i>Android Studio</i>	30
2.9.1 Definition and Overview	30
2.9.2 Key Features.....	30
2.9.3 Application in the Project.....	30
2.9.4 Advantages and Disadvantages	30
2.10 <i>CameraX</i>	31
2.10.1 Definition and Overview	31
2.10.2 Key Features.....	31
2.10.3 Application in the Project.....	31
2.10.4 Advantages and Disadvantages	31
2.11 <i>FastAPI</i>	32
2.11.1 Definition and Overview	32
2.11.2 Key Features.....	32
2.11.3 Application in the Project.....	32
2.11.4 Advantages and Disadvantages	33
2.12 <i>Text-to-Speech (TTS)</i>	33
2.12.1 Definition and Overview	33
2.12.2 Key Features.....	33
2.12.3 Application in the Project.....	33
2.12.4 Advantages and Disadvantages	34
2.13 <i>Haptic Feedback (Vibration)</i>	34
2.13.1 Definition and Overview	34
2.13.2 Key Features.....	35
2.13.3 Application in the Project.....	35
2.13.4 Advantages and Disadvantages	35
2.14 <i>Chapter Summary</i>	36
Chapter 3 - Related Work and Similar Systems	37
3.1 <i>Introduction</i>	37
3.2 <i>Related Work</i>	37
3.2.1 Envision AI	37
3.2.2 Be My Eyes	39
3.3 <i>Similar Systems</i>	40
3.3.1 Envision AI	40
3.3.2 Be My Eyes	41

3.4 Summary	43
Chapter 4 - Analysis	44
4.1 Introduction.....	44
4.2 Functional Requirements	44
4.2.1 User Management	44
4.2.2 Camera and Image Processing.....	44
4.2.3 Object Detection	44
4.2.4 Depth Estimation	45
4.2.5 Risk Assessment	45
4.2.6 User Notification	45
4.2.7 System Management	45
4.3 Non-functional Requirements	45
4.3.1 Performance.....	45
4.3.2 Scalability	45
4.3.3 Security	46
4.3.4 Availability.....	46
4.3.5 Usability.....	46
4.3.6 Reliability.....	46
4.3.7 Maintainability	46
4.3.8 Portability.....	46
4.3.9 Interoperability	47
4.4 UML Diagrams	47
4.5 Summary.....	50
Chapter 5 - Design.....	52
5.1 Introduction.....	52
5.2 Design Alternatives	52
5.2.1 Processing Approach.....	53
5.2.2 Stereo Vision vs. Monocular Depth Estimation	53
5.2.3 Image Processing vs. Object Detection.....	53
5.2.4 Wearable Hardware vs. Mobile Solution	54
5.3 Iterative Design Process.....	54
5.3.1. Requirement Analysis	54
5.3.2. Prototyping	54
5.3.3. System Architecture Design.....	54
5.3.4. Component Design.....	55
5.3.5. Implementation	55
5.3.6. Evaluation and Refinement.....	55
5.4 UML Diagrams	56
5.5. Technologies and Tools Used.....	57
5.5.1. Android Studio	57
5.5.2. YOLO11.....	58
5.5.3. MiDaS	58
5.5.4. OpenCV	58
5.5.5. PyTorch	58
5.5.6. Roboflow	58
5.6. Summary.....	59

Chapter 6 - Implementation	60
6.1. Development Environment	60
6.2. System Architecture Overview	61
6.3 Frontend Implementation	62
6.4. Backend Implementation	63
6.5. AI Integration	66
6.6. Services and Integrations	68
6.7. Challenges Encountered.....	69
6.8. Results and Observations.....	70
6.9. Summary.....	74
Chapter 7 - Testing	75
7.1 Introduction.....	75
7.2 Testing Strategies.....	75
7.2.1 Unit Testing	75
7.2.2 Integration Testing.....	76
7.2.3 System Testing	76
7.2.4 Performance Testing.....	76
7.2.5 Usability Testing.....	77
7.2.6 Reliability Testing	77
7.3 Testing Tools and Environment.....	78
7.4 Test Cases and Results	78
7.4.1 Unit Testing	78
7.4.2 Integration Testing.....	79
7.4.3 System Testing	79
7.4.4 Performance Testing.....	80
7.4.5 Usability Testing.....	80
7.4.6 Reliability Testing	80
7.5 Summary.....	81
Chapter 8 - Conclusion and Future Work.....	82
8.1 Conclusion	82
8.2 Future Work	82
8.2.1 Impact on Visually Impaired Individuals	84
8.2.2 Benefits to Society and Institutions	85
8.2.3 Challenges and Considerations	85
8.2.4 Community Involvement and Collaboration.....	86
8.2.5 Potential for Expansion Beyond Indoor Navigation.....	86
8.2.6 Continuous Improvement and Innovation.....	87
8.2.7 Vision for the Future	87
References.....	88
Appendix A - Software Requirements Specifications IEEE 830.....	90
Appendix B - Software Test Plan IEEE 829	103
Appendix C - Senior Project Summary Report	115

List of Figures

Figure 1-2.2: Flowchart of the Computer Vision Process [19].....	21
Figure 2.3: Example of the typical layered structure of a CNN[20].....	22
Figure 3-2.4:Convolutional Neural Networks (CNNs) [21]	23
Figure 4-2.5: Object Detection work flow[24]	25
Figure 5-2.6: You Only Look Once (YOLO11) [10]	26
Figure 6-2.7: Monocular Depth Estimation [11]	28
Figure 7-2.9 :Android Stodio [14]	31
Figure 8-2.12:Text-to-Speech (TTS) [16].....	34
Figure 9-3.2:Envision AI Application [17].....	38
Figure 10-3.2:Be My Eyes Application [18]	39
Figure 11-4.4: System Architecture Diagram	47
Figure 12-4.4: Sequence Diagram for Obstacle Detection and Alert Generation	48
Figure 13-4.4: Sequence Diagram for Enhanced Visual Processing	49
Figure 14.4 Use Case Diagram	49
Figure 15-5.4: System Workflow	56
Figure 16-5.4: Obstacle Detection Process	57
Figure 17-6.8: Training Performance	70
Figure 18-6.8: Confusion Matrix	71
Figure 19-6.8: Login, Signup Interface.....	72
Figure 20-6.8: Detection And Assistance Interface	73
Figure 21: Test Cases Results	114

List of Tables

Table 1-1.7:SWOT Analysis.....	9
Table 0-1-6.1 Tools And Technologies	61
Table 0-2-6.6: Third party services.....	69
Table 4:Performance Metrics.....	72
Table 0-1-B :Testing schedule	111
Table 0-2-B: bug/issue log.....	113

Abbreviations

CNN	Convolutional Neural Network
YOLO	You Only Look Once
MiDaS	Mixed Dataset Training for Monocular Depth Estimation
API	Application Programming Interface
GUI	Graphical User Interface
IDE	Integrated Development Environment
TTS	Text-to-Speech
UI	User Interface
UX	User Experience
SDK	Software Development Kit
JSON	JavaScript Object Notation
FPS	Frames Per Second
CPU	Central Processing Unit
GPU	Graphics Processing Unit
APK	Android Package Kit
QA	Quality Assurance
UAT	User Acceptance Testing
IEEE	Institute of Electrical and Electronics Engineers
SRS	Software Requirements Specification
STP	Software Test Plan

Chapter 1 - Introduction

Visual impairment is one of the major challenges that significantly affects an individual's independence and quality of life. According to the World Health Organization (WHO), millions of people worldwide suffer from visual impairments that limit their ability to move safely and perform daily activities without assistance. Indoor navigation, in particular, represents a critical challenge because obstacles such as furniture, doors, stairs, and moving individuals can appear unexpectedly and change continuously.

Recent advances in Computer Vision and Deep Learning have enabled the development of intelligent assistive systems capable of understanding surrounding environments using only camera inputs. These technologies can detect objects, estimate distances, and provide immediate feedback to users, making them highly suitable for assistive applications for visually impaired individuals.

This project presents **Smart Vision Assist**, an Android-based assistive application designed to improve indoor mobility and safety for visually impaired users. The application utilizes state-of-the-art Artificial Intelligence techniques, specifically **YOLOv11** for real-time object detection and **MiDaS** for monocular depth estimation. By analyzing live video captured by the smartphone camera, the system identifies indoor obstacles, estimates their relative distances, evaluates potential risks, and provides immediate warnings through speech and vibration feedback.

The proposed solution aims to offer an affordable and practical assistive technology that does not require expensive specialized hardware, relying only on a smartphone equipped with a camera. Consequently, the system can improve user independence and enhance environmental awareness during indoor navigation.

1.1 Motivation

The primary motivation behind this project is the increasing demand for affordable assistive technologies that can improve the quality of life of visually impaired individuals. Existing commercial solutions are often expensive and inaccessible to many users.

Advancements in Artificial Intelligence, Deep Learning, and smartphone computing capabilities make it possible to build practical assistive systems without requiring specialized hardware. Therefore, this project seeks to leverage these technologies to create an intelligent and low-cost solution capable of enhancing indoor mobility and increasing environmental awareness for visually impaired users.

1.2 Problem Statement

Visually impaired individuals encounter significant difficulties when navigating indoor environments independently. Common obstacles such as chairs, tables, stairs, doors, and moving people may cause collisions and accidents, reducing mobility and confidence.

Although several assistive technologies exist, many solutions are expensive, require dedicated wearable devices, or provide limited environmental understanding. Furthermore, some systems lack real-time obstacle detection and distance estimation capabilities that are essential for safe indoor navigation.

Therefore, there is a need for an affordable, intelligent, and portable solution capable of detecting surrounding obstacles, estimating their proximity, and notifying users before potential collisions occur. This project addresses these challenges by developing an Android-based assistive application that combines object detection, depth estimation, and risk assessment techniques to support visually impaired individuals during indoor navigation.

1.3 Objectives

- Develop an Android-based assistive application for visually impaired users.
- Detect indoor obstacles in real time using Computer Vision techniques.
- Estimate the relative distance of detected objects using monocular depth estimation.
- Assess the level of danger associated with surrounding obstacles.
- Provide immediate voice guidance using Text-to-Speech technology.
- Generate vibration alerts to warn users about potential collisions.
- Improve user safety and independent mobility in indoor environments.
- Provide an affordable solution that requires only a smartphone and camera.

1.4 Problem Complexity

The development of Smart Vision Assist involves several technical and operational challenges due to the integration of multiple Artificial Intelligence and mobile technologies within a single application. The system must process live video streams, detect objects in real time, estimate their relative distances, assess risk levels, and provide immediate feedback to users with minimal delay.

The complexity of the project arises from several factors. First, the object detection model must accurately identify various indoor obstacles under different lighting conditions and viewing angles. Second, monocular depth estimation is inherently challenging because distance information must be inferred from a single camera image without dedicated depth sensors. Third, the system requires the integration of multiple components, including Computer Vision models, Android application modules, speech synthesis services, and haptic feedback mechanisms, while maintaining real-time performance on mobile devices.

Moreover, the application is intended for visually impaired users, making reliability and responsiveness critical requirements. Delayed or inaccurate warnings could negatively affect user safety. Therefore, achieving a balance between detection accuracy, computational efficiency, and usability significantly increases the overall complexity of the proposed system.

1.5 Constraints

The development of **Smart Vision Assist** was subject to several technical, operational, and project-related constraints that influenced both the design decisions and implementation process of the proposed system.

Technical Constraints

- **Limited Mobile Processing Resources:**

The application is intended to run on Android smartphones, which possess significantly lower computational capabilities than high-performance workstations and servers. Consequently, achieving real-time object detection and depth estimation while maintaining acceptable performance posed a considerable challenge.

- **Real-Time Processing Requirements:**

Since the application is designed to assist visually impaired users during navigation, the system must process camera frames, perform inference, assess risks, and generate warnings within a very short period. Any noticeable delay may negatively affect the usability and safety of the system.

- **Monocular Vision Limitation:**

The proposed solution relies solely on a single smartphone camera without the use of dedicated depth sensors such as LiDAR or stereo cameras. Therefore, accurate distance estimation becomes inherently challenging and depends entirely on deep learning-based monocular depth estimation techniques.

- **Variations in Environmental Conditions:**

Indoor environments may differ significantly in terms of lighting conditions, object arrangements, occlusions, and viewing perspectives. These variations directly affect detection accuracy and depth estimation performance.

- **Dataset Limitations:**

Obtaining a sufficiently large and diverse indoor dataset containing all relevant obstacle categories and scenarios was difficult. Dataset limitations may reduce the generalization capability of the trained models.

- **Model Size and Memory Consumption:**

Deep learning models require considerable memory and computational resources. Therefore, model selection and optimization were necessary to ensure that the application could operate efficiently on mobile devices.

Project Constraints

- **Limited Development Time:**

The project was developed within a single academic semester, which imposed restrictions on the number of features that could be designed, implemented, and thoroughly evaluated.

- **Limited Hardware Resources:**

The project relied primarily on personal computers and smartphones available to the development team. The absence of specialized hardware constrained experimentation and large-scale model training activities.

1.6 Standards

The development of Smart Vision Assist followed several internationally recognized standards and best practices to ensure quality, reliability, accessibility, and maintainability throughout the project lifecycle:

- **IEEE 12207 – Software Life Cycle Processes:** Adopted to provide a structured framework for software development activities, including planning, requirements analysis, design, implementation, testing, and maintenance [1].
- **IEEE 830 – Software Requirements Specification (SRS):** Used to document and organize the system's functional and non-functional requirements in a clear and standardized format, as presented in Appendix A [2].
- **IEEE 829 – Software Test Documentation:** Applied to structure the software testing process, including test planning, test cases, defect reporting, and test summaries, as detailed in Appendix B.
- **UML Standards (Unified Modeling Language):** UML diagrams, including Use Case Diagrams, Sequence Diagrams, System Architecture Diagrams, and System Workflow Diagrams, were utilized to model the system behavior, interactions, and architecture, as presented in Chapter 4 and Chapter 5.
- **ISO 40500:2012 – Web Content Accessibility Guidelines (WCAG 2.0):** Considered during the design process to promote accessibility and usability. The system incorporates user-friendly interfaces, voice notifications, and vibration feedback mechanisms to support visually impaired users and ensure an inclusive user experience [3].
- **Ethical AI Guidelines:** Best practices for responsible Artificial Intelligence development were considered to ensure transparency, reliability, fairness, and

protection of user data during the processing of visual information.

By adhering to these standards and guidelines, the development of Smart Vision Assist remained systematic, verifiable, and aligned with industry and accessibility requirements, resulting in a reliable assistive solution for visually impaired individuals during indoor navigation.

1.7 Feasibility Study and Business Canvas

Smart Vision Assist is an Android-based assistive application designed to improve indoor mobility and environmental awareness for visually impaired individuals. The system leverages Artificial Intelligence techniques, including real-time object detection and monocular depth estimation, to identify obstacles, estimate their relative distances, and provide immediate warning notifications through speech and vibration feedback.

The proposed solution is technically and economically feasible because it relies primarily on smartphone hardware and open-source technologies, eliminating the need for expensive specialized devices. The system can therefore provide an affordable and accessible assistive solution that can be easily adopted by users and supporting organizations.

Our Offer

Introduce the application as an affordable smartphone-based assistive solution that provides real-time obstacle detection and navigation awareness for visually impaired individuals. A demonstration version of the application can be provided to educational institutions and rehabilitation centers, allowing stakeholders to evaluate the system's capabilities before large-scale deployment and adoption.

Target Markets

- Visually Impaired Individuals

Provide an accessible and low-cost assistive application that increases independent mobility, improves environmental awareness, and reduces the risk of collisions during indoor navigation.

- Educational Institutions

Support schools and universities that serve visually impaired students by providing an affordable technological solution that assists students in navigating classrooms, laboratories, libraries, and campus facilities more safely and independently.

- Rehabilitation and Healthcare Centers

Provide rehabilitation centers and organizations supporting visually impaired individuals with a practical assistive tool that can complement existing training programs and mobility assistance services.

Demand Generation

- Comprehensive Documentation

Provide detailed documentation describing the system architecture, Artificial Intelligence models, risk assessment mechanism, and accessibility features to demonstrate the functionality and effectiveness of the proposed solution.

- Content Marketing

Develop presentations, reports, demonstration videos, and technical publications that explain the challenges faced by visually impaired individuals and illustrate how Artificial Intelligence technologies can improve their daily mobility and safety.

- Website Materials

Problem and Solution

Present the difficulties associated with indoor navigation for visually impaired individuals, including obstacle awareness and collision risks, and introduce Smart Vision Assist as an affordable smartphone-based solution that utilizes Computer Vision and Artificial Intelligence to provide real-time assistance.

- Demonstration Videos

Provide demonstration videos illustrating the application's functionalities, including object detection, depth estimation, risk assessment, and warning notifications under various indoor scenarios.

- SWOT Analysis

A SWOT analysis was conducted to identify the platform's internal strengths and weaknesses, as well as external opportunities and threats. The summary is presented in Table 1.7.

Table 1-1.7:SWOT Analysis

Strengths • Provides real-time obstacle detection and environmental awareness. • Requires only a smartphone camera without specialized hardware. • Combines object detection and depth estimation for better scene understanding. • Offers multimodal feedback through speech and vibration alerts. • Portable, affordable, and easy to deploy on mobile devices. • Enhances independence and safety for visually impaired users during indoor navigation.	Weaknesses • Relative depth estimation is less accurate than dedicated depth sensors. • Performance may decrease under poor lighting conditions. • Real-time AI processing can consume significant battery and computational resources. • Detection accuracy depends on the quality and diversity of the training dataset. • Very small or partially occluded objects may be difficult to detect. • Frequent notifications may overwhelm users in complex environments.
Opportunities • Increasing global demand for assistive technologies for people with disabilities. • Potential integration with wearable devices and smart glasses in future versions. • Expansion to outdoor navigation and smart city applications.	Threats • Rapid evolution of AI technologies may require continuous model updates. • Competition from commercial assistive navigation applications and devices.

• Integration with IoT systems and indoor positioning technologies. • Collaboration opportunities with healthcare institutions and accessibility organizations. • Adoption in educational institutions, hospitals, and public facilities.	• Hardware limitations may affect performance on low-end smartphones. • Variations in indoor environments may reduce model generalization performance. • Dependency on smartphone camera quality and operating system compatibility.
---	--

Future Plans

- **Partnerships**

Collaborate with educational institutions, rehabilitation centers, accessibility organizations, and healthcare providers to evaluate the application in real-world environments and promote its adoption among visually impaired communities.

- **Tactics for Reaching Stakeholders**

- Direct communication with educational institutions and rehabilitation centers.
- Presentations and demonstrations during conferences and academic exhibitions.
- Participation in accessibility and assistive technology events.
- Collaboration with organizations supporting visually impaired individuals.
- Publication of demonstration videos and project materials through websites and social media platforms.
- Development of partnerships with healthcare providers and accessibility initiatives to facilitate application deployment and continuous improvement.

1.8 Thesis Organization

This thesis is organized into chapters as follows:

- Chapter 1 – Introduction: Presents the project motivation, problem statement, objectives, problem complexity, project constraints, applicable standards, feasibility study, and an overview of the thesis organization.
- Chapter 2 – Background: Discusses the theoretical foundations of computer vision, assistive technologies for visually impaired individuals, object detection techniques, monocular depth estimation, and the artificial intelligence technologies utilized in the project, including YOLOv11 and MiDaS.
- Chapter 3 – Related Work and Similar Systems: Reviews existing assistive technologies and mobile applications for visually impaired users, including systems such as Envision AI and Be My Eyes, and analyzes their functionalities, technologies, advantages, and limitations.
- Chapter 4 – Analysis: Presents the functional and non-functional requirements of the Smart Vision Assistive System and includes UML diagrams that describe the system architecture, user interactions, and system workflows.
- Chapter 5 – Design: Describes the design alternatives considered during development, the iterative design process followed throughout the project, and the technologies and tools used to design the mobile-based assistive system.
- Chapter 6 – Implementation: Explains the implementation environment, software architecture, development tools, system components, and the integration of object detection and depth estimation models within the Android application.
- Chapter 7 – Testing: Describes the testing methodologies, testing environment, test cases, and evaluation results used to validate the functionality, usability, performance, and reliability of the proposed system.
- Chapter 8 – Conclusion and Future Work: Summarizes the project contributions and achievements, discusses current limitations, and proposes potential enhancements and future research directions for improving intelligent assistive technologies for visually impaired individuals.

- Appendices: Include supplementary materials such as the Software Requirements Specification (IEEE 830), Software Test Documentation (IEEE 829), UML diagrams, and the Senior Project Summary Report

Chapter 2 – Background

2.1 Introduction

This chapter presents the technologies and tools used in the development of the Smart Vision Assist system. Understanding the theoretical and technical foundations of these technologies is essential for appreciating how the proposed application assists visually impaired individuals during indoor navigation.

The proposed system integrates Artificial Intelligence, Computer Vision, and mobile technologies to provide real-time environmental awareness and obstacle avoidance assistance. The application captures live video streams, detects surrounding objects, estimates their relative distances, evaluates potential risks, and provides immediate warning notifications through speech and vibration feedback.

The technologies and tools utilized in this project include:

1. Computer Vision
2. Deep Learning
3. Convolutional Neural Networks (CNNs)
4. Object Detection
5. YOLO11
6. Monocular Depth Estimation
7. MiDaS
8. Android Studio
9. CameraX
10. FastAPI
11. Text-to-Speech (TTS)
12. Haptic Feedback (Vibration)

Each technology plays an essential role in the functionality and performance of the proposed system. The following sections discuss these technologies in detail and explain their contribution to the development of Smart Vision Assist.

2.2 Computer Vision

2.2.1 Definition and Overview

Computer Vision is a branch of Artificial Intelligence that enables computers to acquire, process, analyze, and interpret visual information from images and videos. The primary objective of Computer Vision is to imitate the human visual system by allowing machines to understand their surrounding environments and make intelligent decisions based on visual data.

Computer Vision combines concepts from image processing, machine learning, deep learning, and pattern recognition to extract meaningful information from visual inputs. Modern Computer Vision systems are capable of performing complex tasks such as object detection, image classification, image segmentation, facial recognition, and scene understanding.

2.2.2 Key Features

- Automatic extraction of visual information from images and videos.
- Real-time image processing capabilities.
- Ability to recognize and classify objects.
- Scene understanding and environmental analysis.
- Integration with Artificial Intelligence and Deep Learning techniques.

2.2.3 Application in the Project

Computer Vision represents the fundamental technology behind Smart Vision Assist. The application continuously analyzes live video streams captured by the smartphone camera to detect indoor obstacles, estimate their relative distances, and provide warning notifications to visually impaired users. Computer Vision techniques allow the system to understand the surrounding environment and improve navigation safety.

2.2.4 Advantages and Disadvantages

Advantages

- Enables automated environmental understanding.
- Supports real-time obstacle detection.
- Provides rich information from visual data.
- Can operate using ordinary smartphone cameras.

Disadvantages

- Sensitive to lighting conditions and image quality.
- Computationally intensive for real-time applications.
- Performance depends heavily on the quality of training datasets.

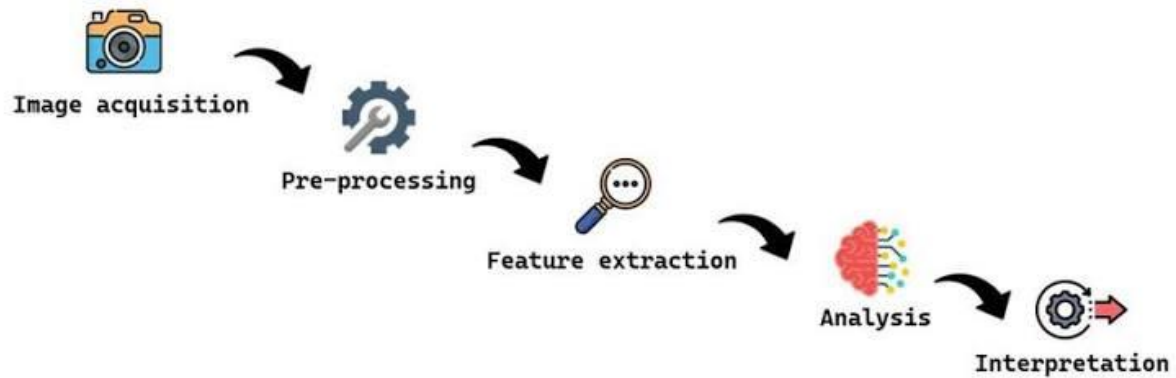


Figure 1-2.2: Flowchart of the Computer Vision Process [19]

2.3 Deep Learning

2.3.1 Definition and Overview

Deep Learning is a specialized field of Machine Learning that utilizes artificial neural networks with multiple hidden layers to learn complex patterns and representations from large amounts of data. Unlike traditional machine learning techniques that require manual feature extraction, Deep Learning models automatically learn hierarchical features directly from raw data.

Deep Learning has significantly improved the performance of Computer Vision applications and has become the foundation of modern object detection, image classification, and depth estimation systems [4].

2.3.2 Key Features

- Automatic feature extraction.
- Ability to learn complex patterns.
- High accuracy in image recognition tasks.
- Scalability with large datasets.
- End-to-end learning capabilities.

2.3.3 Application in the Project

Deep Learning techniques are extensively used in Smart Vision Assist . YOLO11 utilizes deep neural networks to detect indoor obstacles, while MiDaS employs deep learning architectures to estimate the relative depth of objects using monocular images. These technologies collectively enable real-time environmental understanding.

2.3.4 Advantages and Disadvantages

Advantages

- High accuracy in visual recognition tasks.
- Learns features automatically.
- Excellent performance with large datasets.

Disadvantages

- Requires significant computational resources.
- Training can be time-consuming.
- Performance depends on data quality and diversity.

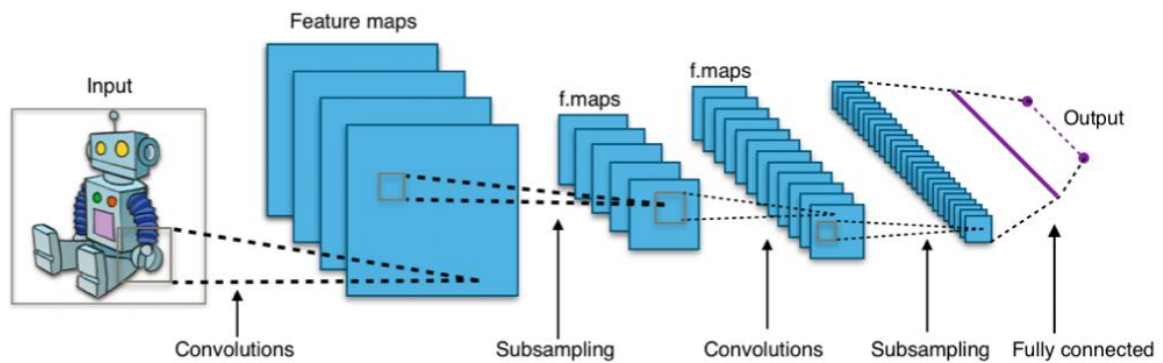


Figure 2.3: Example of the typical layered structure of a CNN[20]

2.4 Convolutional Neural Networks (CNNs)

2.4.1 Definition and Overview

Convolutional Neural Networks (CNNs) are a class of Deep Learning models specifically designed for processing visual data. CNNs employ convolution operations to automatically extract spatial features such as edges, textures, shapes, and object components from images [6].

CNN architectures have become the backbone of most modern Computer Vision applications due to their ability to achieve high accuracy while reducing the need for manual feature engineering.

2.4.2 Key Features

- Automatic extraction of image features.
- Parameter sharing and reduced computational complexity.
- Translation invariance.
- Hierarchical feature learning.
- Efficient image representation.

2.4.3 Application in the Project

CNNs constitute the core computational mechanism behind both YOLO11 and MiDaS. The models utilize convolutional operations to identify obstacles, understand scene structures, and estimate relative object distances.

2.4.4 Advantages and Disadvantages

Advantages

- High accuracy for visual tasks.
- Efficient image feature extraction.
- Excellent performance in object detection.

Disadvantages

- Requires large training datasets.
- Computationally demanding.
- Model performance may decrease under challenging environmental conditions.

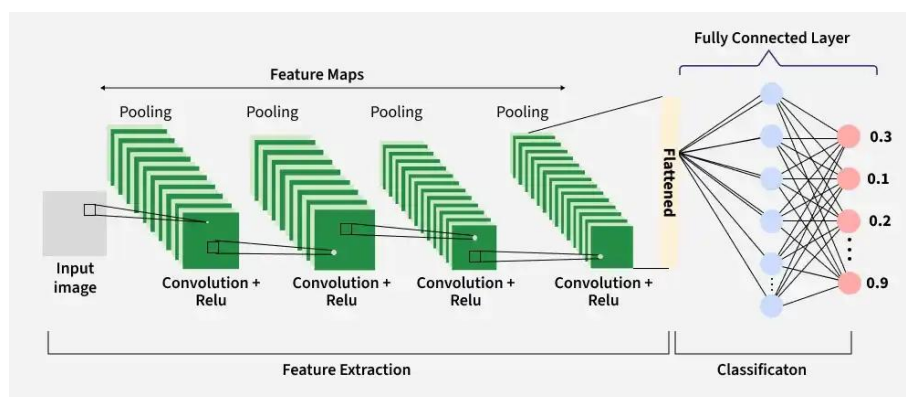


Figure 3-2.4: Convolutional Neural Networks (CNNs) [21]

2.5 Object Detection

2.5.1 Definition and Overview

Object Detection is a Computer Vision task that involves identifying objects within an image and determining their precise locations using bounding boxes and class labels [7]. Unlike image classification, object detection provides both object recognition and localization simultaneously.

Object detection systems are widely used in autonomous systems, surveillance applications, robotics, and assistive technologies.

2.5.2 Key Features

- Object localization.
- Object classification.
- Multi-object detection capabilities.
- Real-time processing.
- Scene understanding.

2.5.3 Application in the Project

Object detection is the primary functionality of Smart Vision Assist (SVAG). The application detects indoor obstacles such as chairs, tables, stairs, doors, walls, and people, allowing the system to determine which objects may pose potential risks to visually impaired users.

2.5.4 Advantages and Disadvantages

Advantages

- Provides precise obstacle localization.
- Supports real-time decision-making.
- Enables scene understanding.

Disadvantages

- Sensitive to occlusions and lighting changes.
- Requires substantial computational resources.

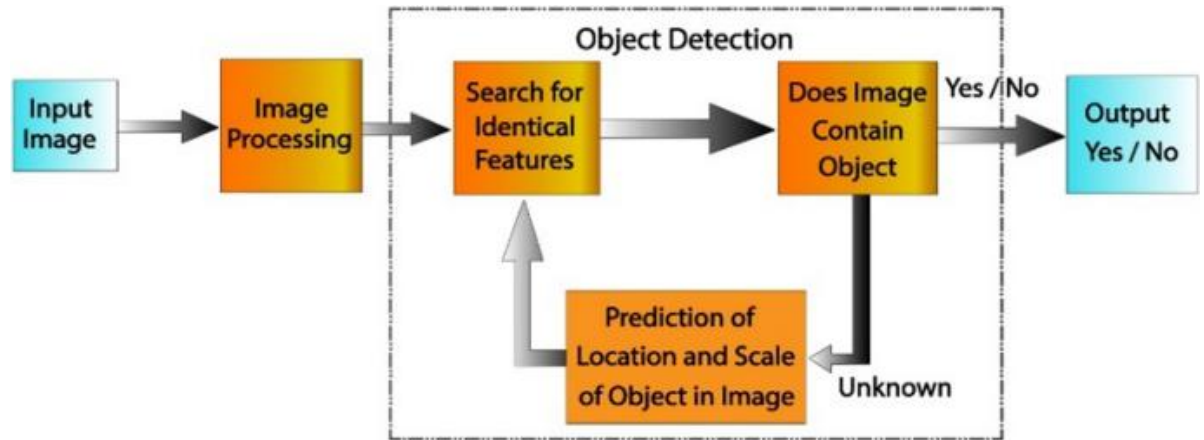


Figure 4-2.5: Object Detection work flow[24]

2.6 You Only Look Once (YOLO11)

2.6.1 Definition and Overview

You Only Look Once (YOLO) is a state-of-the-art object detection algorithm that performs object localization and classification in a single forward pass through a neural network. Unlike traditional object detection methods that rely on multiple stages, YOLO treats object detection as a single regression problem, enabling high-speed and accurate detection.

YOLO11 is one of the latest versions of the YOLO family developed by Ultralytics. It introduces improvements in feature extraction, network efficiency, and detection accuracy while maintaining real-time performance. The model is capable of detecting multiple objects simultaneously and performs effectively in complex scenes containing objects of different sizes and positions.

Due to its high speed and efficiency, YOLO11 has become widely adopted in applications that require real-time decision-making, including autonomous systems, surveillance systems, robotics, and assistive technologies [9].

2.6.2 Key Features

- Real-time object detection capabilities.
- Simultaneous object localization and classification.
- Detection of multiple objects within a single image.
- High detection accuracy with low latency.
- Efficient computational performance.
- Scalability for different hardware platforms.
- Support for custom dataset training and transfer learning.

2.6.3 Application in the Project

In Smart Vision Assist, YOLO11 serves as the primary object detection engine. The model was trained on a custom indoor dataset containing twenty obstacle categories commonly encountered during indoor navigation, including chairs, tables, stairs, doors, walls, windows, and people.

During execution, YOLO11 continuously processes frames captured by the smartphone camera and identifies surrounding objects by generating bounding boxes and class labels. The detected objects are then forwarded to the risk assessment module, where their relative distances and potential hazards are evaluated.

The use of YOLO11 enables the system to provide real-time obstacle awareness while maintaining an acceptable level of computational efficiency for mobile deployment.

2.6.4 Advantages and Disadvantages

Advantages

- Extremely fast inference speed.
- High object detection accuracy.
- Supports real-time applications.
- Efficient utilization of computational resources.
- Easily adaptable to custom datasets.

Disadvantages

- Performance may degrade under poor lighting conditions.
- Small and heavily occluded objects may be difficult to detect.
- Requires a sufficiently diverse dataset to achieve good generalization.

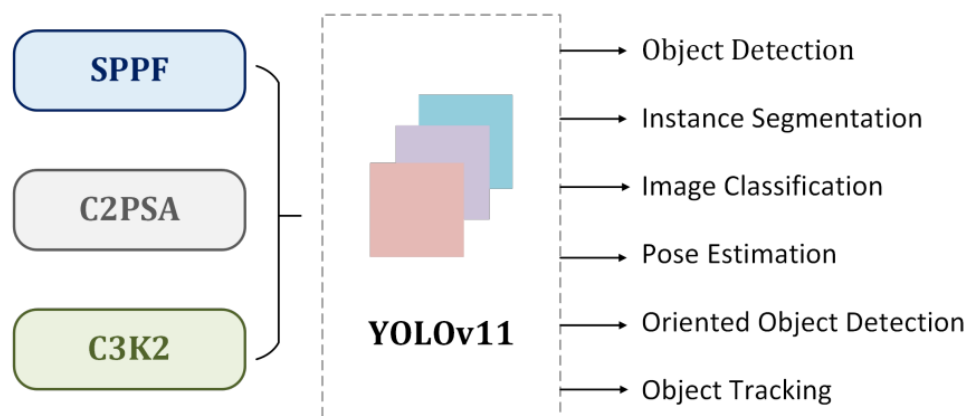


Figure 5-2.6: You Only Look Once (YOLO11) [10]

2.7 Monocular Depth Estimation

2.7.1 Definition and Overview

Depth estimation is the process of determining the distance between a camera and objects present in a scene. Traditional depth estimation approaches often rely on stereo cameras or dedicated depth sensors to obtain three-dimensional information.

Monocular Depth Estimation aims to infer depth information using only a single RGB image. Recent advances in Deep Learning have significantly improved the accuracy of monocular depth estimation by enabling neural networks to learn complex spatial relationships and scene structures.

Monocular depth estimation is particularly attractive for mobile applications because it eliminates the need for expensive hardware while still providing valuable information about object proximity.

2.7.2 Key Features

- Uses only a single RGB image.
- Eliminates the need for additional sensors.
- Provides relative distance estimation.
- Suitable for mobile and low-cost applications.
- Can operate in real-time environments.

2.7.3 Application in the Project

Object detection alone cannot determine how close an obstacle is to the user. Therefore, Smart Vision Assist employs monocular depth estimation techniques to estimate the relative distances of detected objects.

After objects are detected using YOLO11, depth estimation is performed on the same camera frame. The estimated depth information is then combined with object classes to calculate risk scores and determine whether warning notifications should be generated.

2.7.4 Advantages and Disadvantages

Advantages

- Requires only a smartphone camera.
- Low-cost implementation.
- Easy integration with Computer Vision systems.
- Suitable for portable applications.

Disadvantages

- Provides relative rather than absolute distances.
- Sensitive to image quality and lighting conditions.
- Lower accuracy compared to dedicated depth sensors.

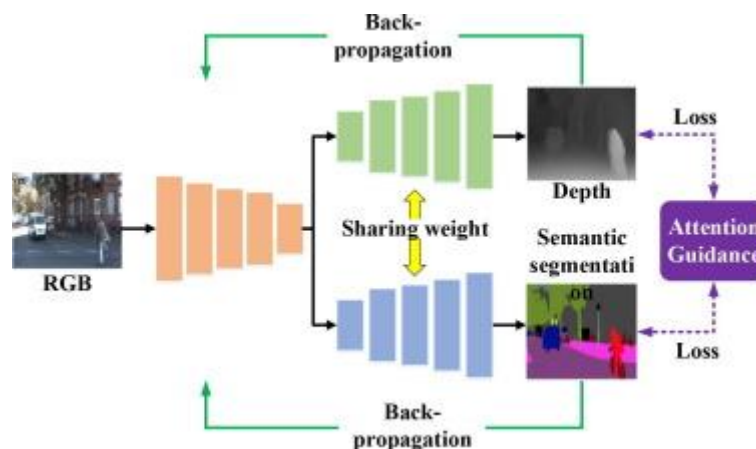


Figure 6-2.7: Monocular Depth Estimation [11]

2.8 Monocular Depth Estimation at Scale (MiDaS)

2.8.1 Definition and Overview

MiDaS (Mixed Dataset Training for Monocular Depth Estimation) is a Deep Learning model developed for estimating scene depth from a single RGB image. The model was trained using multiple datasets to improve generalization across different environments and image characteristics.

MiDaS utilizes deep neural network architectures to infer depth relationships and generate dense depth maps that represent the relative distances of objects within a scene. The model has demonstrated strong performance across various indoor and outdoor environments and is widely used in Computer Vision applications requiring depth perception without specialized hardware [12].

2.8.2 Key Features

- Monocular depth estimation from RGB images.
- Dense pixel-level depth prediction.
- High generalization capability.
- Compatibility with various environments.
- Efficient integration with Computer Vision applications.

2.8.3 Application in the Project

In Smart Vision Assist, MiDaS is responsible for generating depth maps from live camera frames. The depth map is analyzed together with the detected objects produced by YOLO11.

The center point of each detected object's bounding box is mapped onto the generated depth map to estimate its relative distance from the user. This information is subsequently utilized by the risk assessment module to determine the level of danger associated with each obstacle and trigger appropriate warning notifications.

2.8.4 Advantages and Disadvantages

Advantages

- Requires only a single camera.
- No specialized depth sensors are needed.
- Good generalization performance.
- Suitable for mobile and assistive applications.

Disadvantages

- Produces relative rather than exact metric distances.
- Computationally intensive for low-end devices.
- Performance may vary depending on scene complexity.

2.9 Android Studio

2.9.1 Definition and Overview

Android Studio is the official Integrated Development Environment (IDE) for Android application development. It provides a comprehensive set of tools for designing user interfaces, managing application resources, debugging applications, and integrating third-party libraries.

Android Studio supports Java and Kotlin programming languages and offers advanced development features, including code completion, performance profiling, and emulator support.

2.9.2 Key Features

- Official Android development environment.
- User interface design tools.
- Emulator and debugging capabilities.
- Dependency management.
- Integration with Android SDK libraries.

2.9.3 Application in the Project

Android Studio was used to develop the Smart Vision Assist application. The platform was utilized to design the user interface, integrate camera services, implement speech and vibration notifications, and establish communication with backend services [13].

2.9.4 Advantages and Disadvantages

Advantages

- Comprehensive development environment.
- Excellent debugging tools.
- Large developer community.
- Extensive documentation and support.

Disadvantages

- Requires significant system resources.
- Emulator performance can be demanding on low-specification machines.



Figure 7-2.9 :Android Stodio [14]

2.10 CameraX

2.10.1 Definition and Overview

CameraX is a Jetpack library developed by Google to simplify camera integration in Android applications. It provides a lifecycle-aware API that reduces development complexity and ensures compatibility across different Android devices.

2.10.2 Key Features

- Simplified camera integration.
- Lifecycle-aware design.
- Device compatibility.
- Real-time image capture and analysis.

2.10.3 Application in the Project

CameraX was used to capture live video streams from the smartphone camera and provide image frames for processing by the Artificial Intelligence modules.

2.10.4 Advantages and Disadvantages

Advantages

- Easy implementation.
- Stable camera performance.
- Wide device compatibility.

Disadvantages

- Limited low-level camera control compared to Camera2 API.

2.11 FastAPI

2.11.1 Definition and Overview

FastAPI is a modern, high-performance web framework for building Application Programming Interfaces (APIs) using Python. It was developed to simplify the creation of RESTful services while maintaining excellent performance and scalability.

FastAPI is based on standard Python type hints and automatically generates interactive API documentation, making it easier to develop, test, and maintain backend services. Due to its lightweight architecture and high execution speed, FastAPI has become widely adopted in Artificial Intelligence and Machine Learning applications that require communication between mobile applications and computational models [15].

2.11.2 Key Features

- High-performance API development.
- Asynchronous request handling.
- Automatic generation of interactive API documentation.
- Simple and readable code structure.
- Easy integration with Machine Learning and Deep Learning models.
- Support for scalable and maintainable system architectures.

2.11.3 Application in the Project

In Smart Vision Assist , FastAPI acts as the communication layer between the Android application and the Artificial Intelligence modules. The Android application sends captured image frames to the backend service, where object detection and depth estimation models process the data.

After processing, the backend returns the detected objects, estimated distances, and risk levels to the Android application, which subsequently generates voice and vibration notifications for the user. The use of FastAPI enables efficient data exchange and facilitates the integration of AI components with the mobile application.

2.11.4 Advantages and Disadvantages

Advantages

- High execution speed and low latency.
- Easy integration with Python-based AI models.
- Automatically generated API documentation.
- Simple development and maintenance.
- Excellent scalability.

Disadvantages

- Relatively newer framework compared to traditional web frameworks.
- Requires knowledge of asynchronous programming concepts for advanced implementations.

2.12 Text-to-Speech (TTS)

2.12.1 Definition and Overview

Text-to-Speech (TTS) is an assistive technology that converts textual information into synthesized speech. TTS systems analyze input text and generate audible output that allows users to receive information through spoken language instead of visual interfaces.

TTS technologies are widely used in accessibility applications, navigation systems, educational software, and digital assistants. For visually impaired users, speech feedback serves as one of the most important interaction mechanisms because it enables hands-free and eyes-free communication with technological systems.

2.12.2 Key Features

- Converts text into spoken language.
- Provides real-time auditory feedback.
- Supports accessibility and assistive applications.
- Allows hands-free interaction.
- Supports multiple languages and voice configurations.

2.12.3 Application in the Project

Text-to-Speech functionality is one of the primary feedback mechanisms in Smart Vision Assist. After the risk assessment module determines that an obstacle poses a potential danger, the application generates verbal notifications describing the detected obstacle and its risk level.

Examples of warning messages include:

- "Warning, person ahead."
- "Caution, stairs detected."
- "Obstacle nearby."

The auditory feedback enables visually impaired users to receive immediate environmental information without requiring visual attention.

2.12.4 Advantages and Disadvantages

Advantages

- Improves accessibility.
- Provides immediate feedback.
- Enables hands-free interaction.
- Reduces dependency on visual interfaces.

Disadvantages

- Speech may be difficult to hear in noisy environments.
- Excessive notifications may overwhelm users.
- Audio feedback may raise privacy concerns in public spaces.

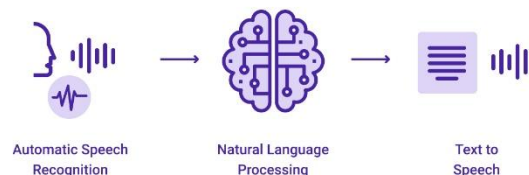


Figure 8-2.12:Text-to-Speech (TTS) [16]

2.13 Haptic Feedback (Vibration)

2.13.1 Definition and Overview

Haptic Feedback refers to the use of tactile sensations, such as vibrations, to communicate information to users. Modern smartphones incorporate vibration motors that can generate different patterns of tactile feedback to indicate events, warnings, or user interactions.

Haptic technologies have become an important component of accessibility systems because they provide non-visual and non-auditory communication methods that can complement other assistive mechanisms.

2.13.2 Key Features

- Tactile and non-visual communication.
- Immediate user notification.
- Low cognitive load.
- Ability to complement auditory feedback.
- Effective in noisy environments.

2.13.3 Application in the Project

In Smart Vision Assist , vibration notifications are used together with speech warnings to alert users about nearby obstacles and potential hazards.

The risk assessment module determines the severity of each detected obstacle and triggers vibration feedback accordingly. The use of haptic notifications ensures that warnings can still be perceived even when speech notifications are difficult to hear.

Combining vibration and speech notifications improves system reliability and provides multimodal feedback for visually impaired users.

2.13.4 Advantages and Disadvantages

Advantages

- Effective in noisy environments.
- Provides immediate feedback.
- Enhances accessibility.
- Consumes relatively low computational resources.
- Complements voice-based notifications.

Disadvantages

- Limited information can be conveyed through vibration alone.
- Continuous vibration may become distracting.
- Different devices may provide varying vibration intensities.

2.14 Chapter Summary

This chapter presented the theoretical and technological foundations of the Smart Vision Assist system. The chapter introduced the concepts of Computer Vision, Deep Learning, Convolutional Neural Networks, and Object Detection, which collectively form the basis of the proposed solution. Furthermore, the chapter discussed the technologies employed during development, including YOLO11 for real-time object detection, MiDaS for monocular depth estimation, Android Studio and CameraX for mobile application development and camera integration, FastAPI for communication between system components, and Text-to-Speech and Haptic Feedback mechanisms for user notifications.

The presented technologies collectively enable Smart Vision Assist to provide real-time obstacle awareness and assist visually impaired individuals in navigating indoor environments more safely and independently. The next chapter introduces the requirements analysis and system specification of the proposed solution.

Chapter 3 - Related Work and Similar Systems

3.1 Introduction

The advancement of Artificial Intelligence and Computer Vision technologies has significantly transformed the development of assistive applications for visually impaired individuals. Modern assistive systems increasingly rely on deep learning algorithms, object recognition techniques, and mobile computing technologies to provide environmental awareness and navigation assistance.

Several commercial applications and research projects have demonstrated the potential of smartphone-based assistive technologies in improving the independence and quality of life of visually impaired users. These systems generally employ image analysis, scene understanding, object recognition, and audio feedback mechanisms to assist users in understanding and interacting with their surroundings.

Recent developments in mobile Artificial Intelligence have enabled the deployment of sophisticated Computer Vision models on smartphones, making assistive technologies more affordable and accessible. Applications such as Envision AI and Be My Eyes represent successful examples of leveraging Artificial Intelligence to provide environmental information and accessibility services to visually impaired individuals.

The proposed Smart Vision Assist (SVAG) application builds upon the concepts demonstrated by existing systems while introducing additional capabilities specifically designed for indoor navigation assistance. By integrating real-time object detection using YOLO11, monocular depth estimation using MiDaS, risk assessment mechanisms, and multimodal feedback through speech and vibration notifications, the proposed solution aims to provide a low-cost and practical indoor navigation assistant that enhances user safety and independence [References].

3.2 Related Work

3.2.1 Envision AI

Envision AI is an Artificial Intelligence-powered assistive application developed specifically for visually impaired individuals. The application utilizes Computer Vision and Optical Character Recognition technologies to recognize text, identify objects, describe scenes, and provide spoken information about the surrounding environment.

The application processes images captured by the smartphone camera and converts visual information into audio descriptions. Envision AI demonstrates the effectiveness of integrating Artificial Intelligence and mobile technologies to improve accessibility and environmental awareness for visually impaired users .

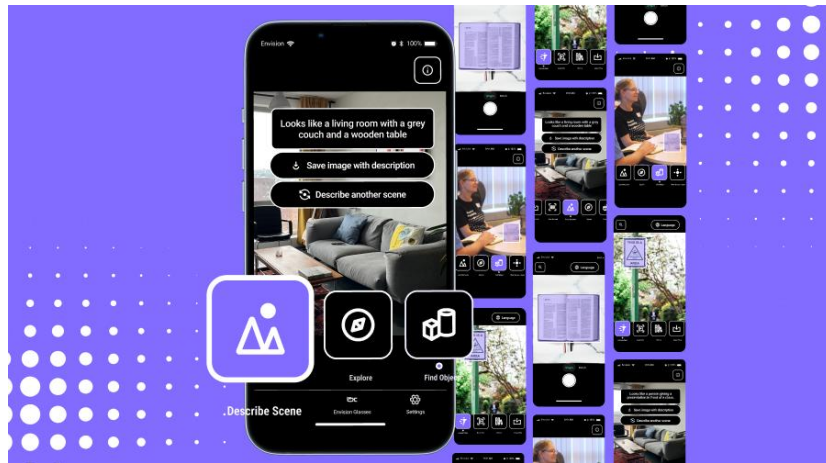


Figure 9-3.2:Envision AI Application [17]

3.2.2 Be My Eyes

Be My Eyes is an assistive application that connects visually impaired users with volunteers and Artificial Intelligence services to provide real-time visual assistance. The application allows users to receive information about objects, read text, and obtain descriptions of their surroundings through live interactions and AI-generated scene interpretations.

The platform illustrates how Artificial Intelligence and human assistance can complement each other in providing accessibility services and improving the independence of visually impaired individuals .

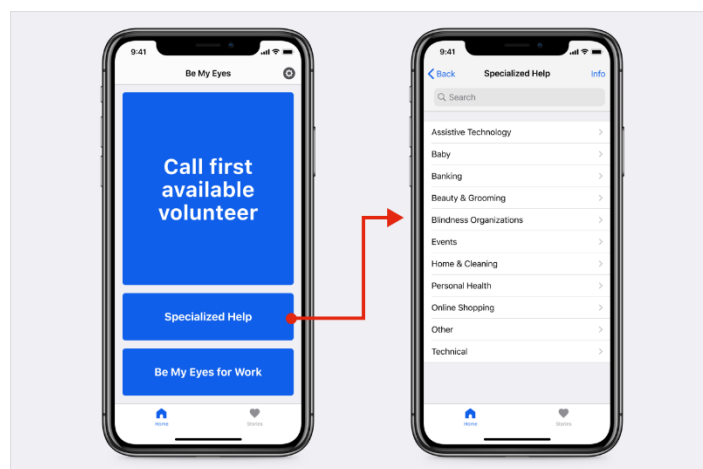


Figure 10-3.2:Be My Eyes Application [18]

3.3 Similar Systems

3.3.1 Envision AI

Overview

Envision AI is a mobile application designed to assist visually impaired users by converting visual information into spoken descriptions. The application supports several accessibility features, including text reading, object recognition, scene description, and person identification.

The system demonstrates how modern Computer Vision and Deep Learning technologies can be integrated into mobile devices to provide environmental awareness and accessibility support.

3.3.1.1 Functionality and Workflow

The primary function of Envision AI is to analyze images captured by the smartphone camera and convert visual information into understandable audio feedback.

Input and Output

Input

- Live camera images
- Printed and handwritten text
- Objects and surrounding scenes

Output

- Spoken scene descriptions
- Object identification results
- Text reading information
- Environmental descriptions

3.3.1.2 Technologies Used

- Artificial Intelligence
- Computer Vision
- Deep Learning
- Optical Character Recognition (OCR)
- Speech Synthesis
- Mobile Computing Technologies

3.3.1.3 Advantages and Disadvantages

Advantages

- Provides rich environmental descriptions.
- Supports multiple accessibility functionalities.
- Operates directly on smartphones.
- Enhances user independence and accessibility.

Disadvantages

- Performance may decrease under poor lighting conditions.
- Limited distance estimation capabilities.
- Primarily focuses on environmental understanding rather than obstacle risk assessment and navigation.

3.3.2 Be My Eyes

Overview

Be My Eyes is an accessibility application developed to assist visually impaired individuals by combining human assistance with Artificial Intelligence technologies. The application allows users to receive information about their surroundings through AI-generated descriptions and real-time support.

The system demonstrates the effectiveness of combining mobile technologies and Artificial Intelligence to provide accessible services and environmental information.

3.3.2.1 Functionality and Workflow

The system captures images or live video streams from the smartphone camera and generates descriptions of surrounding objects and scenes.

Input and Output

Input

- Live camera frames
- Images of surrounding environments
- Objects and textual information

Output

- Spoken scene descriptions
- Object recognition information
- Environmental explanations
- Accessibility assistance services

3.3.2.2 Technologies Used

- Artificial Intelligence
- Computer Vision
- Deep Learning
- Speech Synthesis
- Cloud-Based Services
- Mobile Computing Technologies

3.3.2.3 Advantages and Disadvantages

Advantages

- Provides comprehensive environmental descriptions.
- Combines Artificial Intelligence with accessibility services.
- Easy to use through smartphones.
- Improves independence for visually impaired users.

Disadvantages

- Some functionalities depend on internet connectivity.
- Does not provide dedicated indoor navigation mechanisms.
- Limited obstacle prioritization and risk assessment capabilities.
- Does not perform real-time depth estimation for obstacle avoidance.

3.4 Summary

This chapter reviewed existing Artificial Intelligence-based assistive applications developed for visually impaired individuals. Applications such as Envision AI and Be My Eyes demonstrate the significant potential of integrating Computer Vision, Deep Learning, and mobile technologies to improve accessibility and environmental awareness.

Although these applications provide valuable functionalities, including object recognition, scene description, and text reading capabilities, they exhibit certain limitations regarding indoor navigation support, obstacle prioritization, and real-time distance estimation.

The proposed Smart Vision Assist system addresses these limitations by integrating real-time object detection using YOLO11, monocular depth estimation using MiDaS, risk assessment mechanisms, and multimodal warning notifications through speech and vibration feedback. Furthermore, the proposed solution specifically targets indoor navigation scenarios and aims to provide visually impaired users with safer and more independent mobility experiences.

Chapter 4 - Analysis

4.1 Introduction

This chapter presents a comprehensive analysis of the proposed system, Smart Vision Assist , an Artificial Intelligence-based mobile application designed to assist visually impaired individuals in navigating indoor environments safely and independently.

The system integrates Computer Vision and Deep Learning technologies to detect surrounding obstacles, estimate their relative distances, evaluate potential risks, and provide real-time multimodal notifications through speech and vibration feedback. The application relies solely on smartphone hardware and does not require specialized sensors or expensive wearable devices, making it an affordable and accessible assistive solution.

This chapter analyzes the functional and non-functional requirements of the proposed system and evaluates its expected performance, security, scalability, usability, and maintainability. Furthermore, UML diagrams are presented to illustrate the architecture of the system, user interactions, and the communication between different software components.

The analysis presented in this chapter serves as the foundation for the design and implementation phases discussed in subsequent chapters and ensures that the proposed solution effectively addresses the requirements of visually impaired users.

4.2 Functional Requirements

The **Smart Vision Assist** application shall provide the following functionalities:

4.2.1 User Management

- Launch the application.
- Grant camera permissions.
- Grant microphone and vibration permissions.
- Access application settings.

4.2.2 Camera and Image Processing

- Capture live video frames using the smartphone camera.
- Process video frames in real time.
- Continuously analyze the surrounding environment.

4.2.3 Object Detection

- Detect indoor objects and obstacles.

- Identify object categories.
- Display detected object labels.

4.2.4 Depth Estimation

- Estimate the relative distance of detected objects.
- Continuously update depth information.

4.2.5 Risk Assessment

- Calculate risk scores for detected obstacles.
- Determine obstacle severity levels.
- Prioritize dangerous obstacles.

4.2.6 User Notification

- Generate speech warnings.
- Generate vibration notifications.
- Provide real-time environmental alerts.

4.2.7 System Management

- Start navigation assistance.
- Stop navigation assistance.
- Process continuous video streams.
- Log system events and prediction results.

4.3 Non-functional Requirements

4.3.1 Performance

- The system should process camera frames in near real time.
- The application should provide notifications with minimal latency.
- Object detection and depth estimation should execute efficiently on smartphone hardware.
- The system should maintain acceptable performance during continuous operation.

4.3.2 Scalability

- The system architecture should support future integration of additional Artificial Intelligence models.

- The application should allow future expansion to outdoor navigation scenarios.
- The backend services should support future integration of cloud-based processing if necessary.

4.3.3 Security

- User permissions should be explicitly granted before accessing device hardware.
- Communication between application components should follow secure communication practices.
- User information and application data should be handled securely.
- The application should avoid unnecessary storage of captured images and video streams.

4.3.4 Availability

- The application should remain operational during continuous navigation sessions.
- System services should recover gracefully from temporary failures.
- The application should function without requiring constant internet connectivity.

4.3.5 Usability

- The application interface should be simple and intuitive.
- The system should provide accessible interaction mechanisms suitable for visually impaired users.
- Speech and vibration notifications should be clear and easy to understand.
- User interactions should require minimal manual intervention.

4.3.6 Reliability

- Object detection results should maintain acceptable accuracy in indoor environments.
- Risk notifications should be generated consistently and reliably.
- The system should minimize false alarms and missed obstacle detections.

4.3.7 Maintainability

- The software architecture should be modular and easy to extend.
- Artificial Intelligence modules should be independently maintainable.
- Source code should follow coding standards and documentation practices.
- New obstacle categories and risk assessment rules should be easily integrated.

4.3.8 Portability

- The application should support Android-based smartphones.

- The system should be deployable on different smartphone configurations with minimal modifications.

4.3.9 Interoperability

- The system should allow communication between the Android application and backend Artificial Intelligence services.
- The architecture should support future integration with wearable devices and external assistive technologies.
- APIs should be designed to facilitate future system extensions.

4.4 UML Diagrams

1. System Architecture Diagram

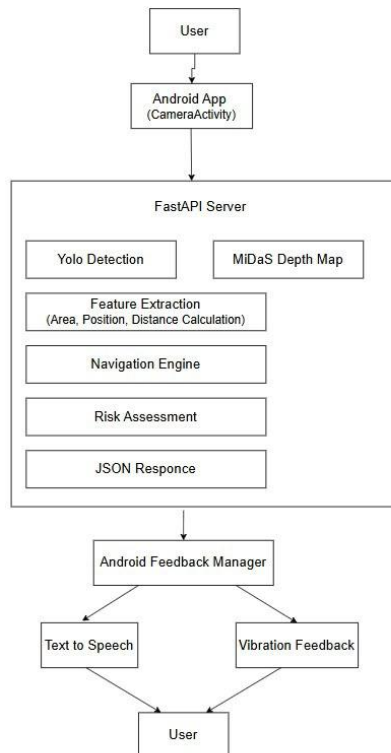


Figure 11-4.4: System Architecture Diagram

The Smart Vision Assist system adopts a client-server architecture consisting of an Android mobile application and an Artificial Intelligence backend server implemented using FastAPI. The architecture integrates Computer Vision and Deep Learning technologies to provide real-time indoor navigation assistance for visually impaired users.

The Android application captures live video streams through the smartphone camera and sends image frames to the FastAPI server for processing. The backend server performs object detection

using the YOLO11 model and depth estimation using the MiDaS model. The extracted information is then analyzed by the Navigation Engine and Risk Assessment module to determine potential hazards and generate suitable warning notifications.

Finally, the processed results are returned to the Android application, where the Feedback Manager generates multimodal notifications through Text-to-Speech and vibration feedback mechanisms.

2. Sequence Diagram: Obstacle Detection and Alert Generation

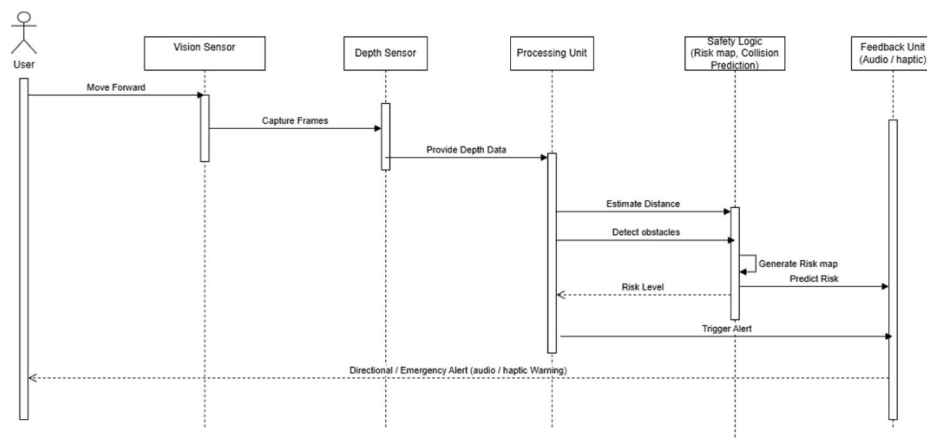


Figure 12-4.4: Sequence Diagram for Obstacle Detection and Alert Generation

The obstacle detection and alert generation process begins when the user starts moving through the indoor environment. The Vision Sensor continuously captures image frames and provides them to the processing unit.

Depth information is then generated and combined with the visual data to estimate object distances and detect surrounding obstacles. The Safety Logic component constructs a risk map and predicts potential collisions based on the detected obstacles and their relative positions.

If a dangerous obstacle is identified, the system triggers appropriate warning notifications through the Feedback Unit. The user subsequently receives directional or emergency alerts in the form of speech messages and vibration feedback.

3. Sequence Diagram: Enhanced Visual Processing

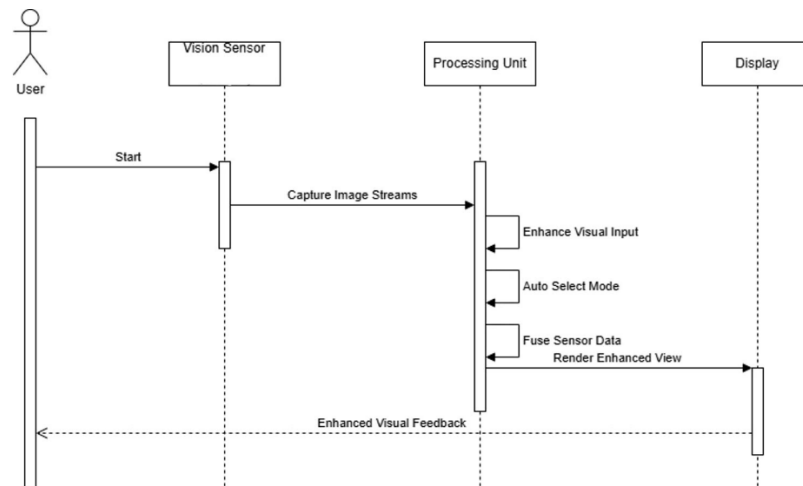


Figure 13-4.4: Sequence Diagram for Enhanced Visual Processing

The enhanced visual processing workflow starts when the user activates the application. The Vision Sensor captures image streams and forwards them to the Processing Unit.

The Processing Unit performs several operations, including:

- Enhancing visual input.
- Automatically selecting the appropriate processing mode.
- Fusing visual information obtained from multiple processing components.

After processing is completed, the enhanced information is rendered and delivered to the display component. The user then receives enhanced visual feedback and environmental awareness information in real time.

4. Use Case Diagram

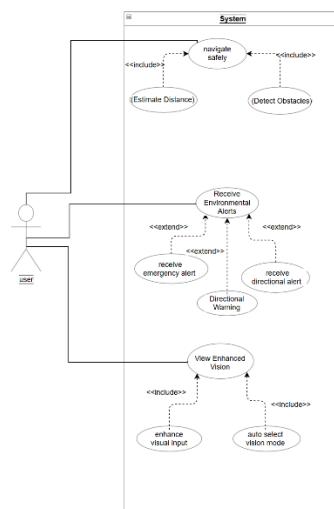


Figure 14.4 Use Case Diagram

The Use Case Diagram illustrates the interactions between the user and the Smart Vision Assist system. The system is designed to assist visually impaired users by providing safe indoor navigation, environmental awareness, and enhanced visual feedback.

The primary actor in the system is the user, who interacts with three major functionalities:

- Navigate Safely
- Receive Environmental Alerts
- View Enhanced Vision

The **Navigate Safely** use case includes obstacle detection and distance estimation functionalities that allow the system to understand the surrounding environment and identify potential hazards.

The **Receive Environmental Alerts** use case is extended by emergency and directional warning services that notify the user whenever dangerous situations are detected.

The **View Enhanced Vision** use case includes visual enhancement mechanisms and automatic mode selection to improve environmental perception and accessibility.

4.5 Summary

This chapter presented a comprehensive analysis of the Smart Vision Assist system, an Artificial Intelligence-based mobile application developed to assist visually impaired individuals during indoor navigation.

The chapter discussed the functional requirements of the system, including real-time image acquisition, obstacle detection, distance estimation, risk assessment, and multimodal feedback through speech and vibration notifications. In addition, several non-functional requirements were identified, including performance, security, reliability, usability, scalability, maintainability, portability, and interoperability, which collectively define the expected operational characteristics of the proposed system.

To provide a clear understanding of the system structure and behavior, several UML and design diagrams were introduced, including:

- Use Case Diagram
- System Architecture Diagram
- Sequence Diagram for Obstacle Detection and Alert Generation
- Sequence Diagram for Enhanced Environmental Processing

These diagrams illustrate the interactions between the user and the system components, the flow of information between the mobile application and the AI processing modules, and the

mechanisms used to detect obstacles, estimate distances, assess risks, and generate real-time notifications.

The analysis demonstrates that Smart Vision Assist integrates modern Computer Vision and Deep Learning technologies to provide an affordable, accessible, and intelligent indoor navigation assistant that improves environmental awareness and promotes safer and more independent mobility for visually impaired users.

The next chapter presents the system design and implementation process of Smart Vision Assist, including the overall system architecture, design decisions, component interactions, and the technologies used to develop the proposed solution.

Chapter 5 - Design

5.1 Introduction

Chapter 5 presents the engineering design and architecture of the **Smart Vision Assist** system. This chapter represents a critical stage in the project, where the requirements and analysis presented in previous chapters are transformed into a practical Artificial Intelligence-based mobile application that assists visually impaired individuals in navigating indoor environments.

The chapter begins by evaluating different design alternatives, including cloud-based versus on-device processing, monocular versus stereo vision approaches, and traditional obstacle detection methods versus deep learning-based Computer Vision techniques. Each alternative is carefully analyzed to ensure that the system satisfies the requirements of affordability, accessibility, accuracy, and real-time performance.

Following this, the chapter describes the iterative design process that guided the development of the application. The process includes requirement analysis, prototyping, system architecture design, component design, implementation, and continuous refinement. These stages were essential in aligning the system with the needs of visually impaired users and ensuring the reliability of the proposed solution.

Additionally, the chapter presents UML diagrams that visually represent the system architecture and interactions among software components. These diagrams provide a clear understanding of how the Smart Vision Assist system operates, beginning with image acquisition and ending with speech and vibration notifications.

Chapter 5 serves as a comprehensive overview of the design decisions and architectural foundations of the proposed system and establishes the basis for the implementation and evaluation stages presented in the following chapter.

5.2 Design Alternatives

During the design of the Smart Vision Assist system, several design alternatives were considered to address both the functional and non-functional requirements of the project.

5.2.1 Processing Approach

Cloud-Based Processing

A cloud-based approach processes captured images on remote servers and returns the detection results to the mobile application. This approach offers high computational capabilities and allows the use of larger Artificial Intelligence models. However, it introduces network latency, depends heavily on internet connectivity, and raises privacy concerns due to the transmission of user data.

On-Device Processing

An on-device approach performs Artificial Intelligence inference directly on the smartphone. This approach provides lower latency, increased privacy, and offline functionality. Since visually impaired users may require immediate responses in various environments, on-device processing was selected as the most suitable approach.

5.2.2 Stereo Vision vs. Monocular Depth Estimation

Stereo Vision

Stereo vision systems use multiple cameras to estimate depth information accurately. Although this approach can achieve reliable distance estimation, it requires specialized hardware and increases system complexity and cost.

Monocular Depth Estimation

Monocular depth estimation uses a single RGB camera and deep learning algorithms to estimate relative object distances. This approach significantly reduces hardware requirements while maintaining acceptable depth estimation performance. Therefore, monocular depth estimation was adopted in this project.

5.2.3 Image Processing vs. Object Detection

Traditional Image Processing

Traditional methods rely on handcrafted features such as edges, contours, and color segmentation. While computationally inexpensive, they generally perform poorly in complex indoor environments and exhibit limited robustness.

Deep Learning-Based Object Detection

Deep learning models automatically learn visual features and achieve significantly higher detection accuracy and robustness under varying conditions. Consequently, deep learning-based object detection was selected as the primary approach for obstacle detection.

5.2.4 Wearable Hardware vs. Mobile Solution

Wearable Hardware

Wearable devices such as smart glasses provide hands-free interaction and may offer additional sensors. However, they significantly increase development costs and may reduce accessibility due to their high price and hardware requirements.

Smartphone-Based Solution

A smartphone-based approach utilizes hardware that users already possess, reducing overall costs and simplifying deployment. Smartphones also provide cameras, processing capabilities, speakers, and vibration mechanisms required by the system. Therefore, a smartphone-based solution was selected.

5.3 Iterative Design Process

The development of Smart Vision Assist followed an iterative engineering approach consisting of several stages of analysis, development, testing, and refinement.

5.3.1. Requirement Analysis

- Identifying the requirements of visually impaired users.
- Defining functional and non-functional requirements.
- Determining system constraints and objectives.

5.3.2. Prototyping

- Developing initial prototypes of the Android application.
- Creating preliminary interfaces and navigation workflows.
- Designing experimental Computer Vision pipelines.

5.3.3. System Architecture Design

- Designing the overall system architecture.
- Defining interactions between the mobile application and Artificial Intelligence modules.
- Ensuring scalability, maintainability, and modularity.

5.3.4. Component Design

- Designing the camera acquisition module.
- Designing the object detection module.
- Designing the depth estimation module.
- Designing the risk assessment module.
- Designing the speech and vibration notification modules.

5.3.5. Implementation

- Developing the Android application.
- Integrating Artificial Intelligence models.
- Implementing the notification mechanisms.
- Performing continuous debugging and optimization.

5.3.6. Evaluation and Refinement

- Evaluating system performance.
- Improving detection reliability.
- Refining risk assessment mechanisms.
- Enhancing user experience and responsiveness.

5.4 UML Diagrams

System Workflow

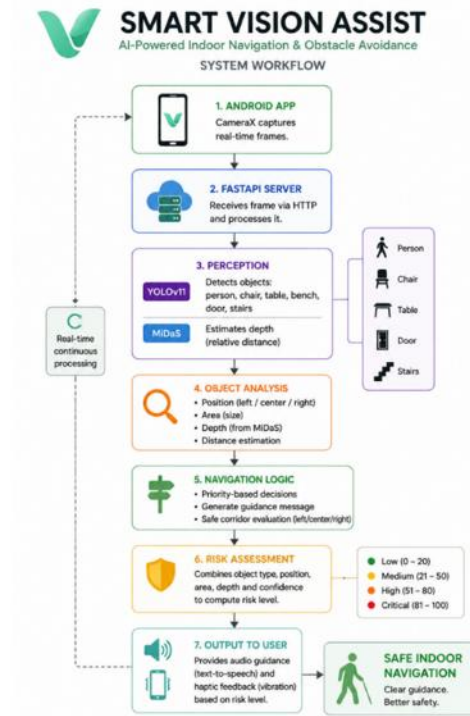


Figure 15-5.4: System Workflow

The System Workflow Diagram shows how Smart Vision Assist processes live camera frames, performs object detection and depth estimation, evaluates environmental risks, and generates real-time speech and vibration notifications to assist visually impaired users during indoor navigation.

Obstacle Detection Process

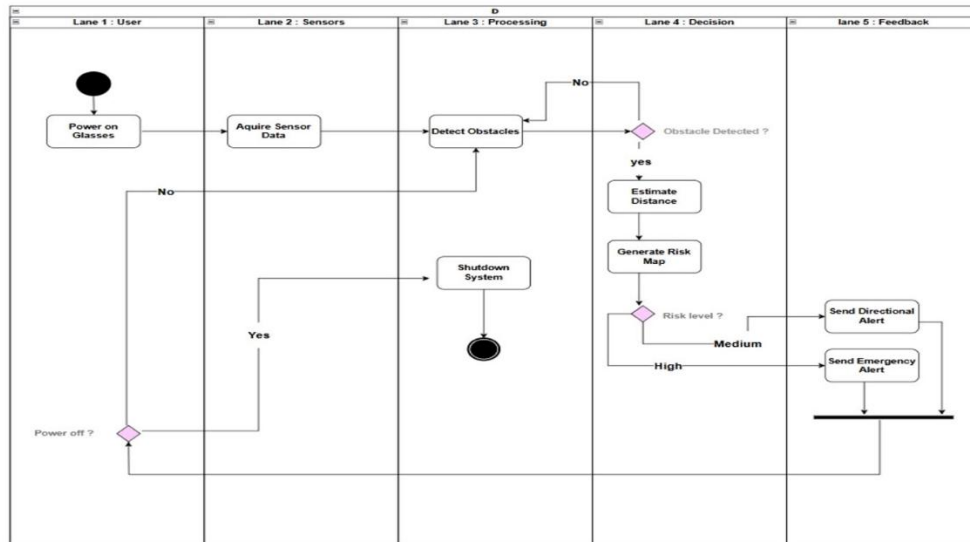


Figure 16-5.4: Obstacle Detection Process

The Activity diagram illustrates the operations involved in detecting obstacles and generating notifications.

The user launches the application and activates navigation assistance. The camera captures video frames and sends them to the Artificial Intelligence modules. The object detection and depth estimation components analyze the frames and provide environmental information to the risk assessment module. Finally, warning notifications are delivered to the user through speech and vibration mechanisms.

5.5. Technologies and Tools Used

The following technologies and tools were utilized during the design and development of Smart Vision Assist.

5.5.1. Android Studio

- Android Studio was used as the primary development environment for building the mobile application.
- It provided tools for interface design, debugging, and application deployment on Android devices.

5.5.2. YOLO11

- YOLO11 was selected as the primary object detection model due to its high detection accuracy and real-time performance.
- The model was trained on a customized indoor dataset containing twenty obstacle categories relevant to indoor navigation scenarios.

5.5.3. MiDaS

- MiDaS was utilized for monocular depth estimation.
- The model estimates relative distances of surrounding objects using images captured by a single RGB camera.
- It eliminated the need for dedicated depth sensors and reduced hardware requirements.

5.5.4. OpenCV

- OpenCV was employed for image processing and frame manipulation.
- It facilitated image acquisition, preprocessing, and visualization of detection results.

5.5.5. PyTorch

- PyTorch was used for loading, training, and executing deep learning models.
- It provided an efficient framework for implementing object detection and depth estimation algorithms.

5.5.6. Roboflow

- Roboflow was used for dataset management, annotation, preprocessing, and dataset organization.
- It simplified the preparation of training, validation, and testing datasets for the object detection model.

5.6. Summary

Chapter 5 presented the engineering design and architecture of the Smart Vision Assist system. The chapter began by evaluating several design alternatives, including cloud-based versus on-device processing, stereo vision versus monocular depth estimation, traditional image processing versus deep learning techniques, and wearable hardware versus smartphone-based solutions.

The chapter also described the iterative development process, which consisted of requirement analysis, prototyping, system architecture design, component design, implementation, and continuous refinement.

Furthermore, UML diagrams were introduced to illustrate the system architecture and interactions among software components. The technologies and tools employed during development, including Android Studio, Java, YOLO11, MiDaS, OpenCV, PyTorch, and Roboflow, were also discussed.

The engineering decisions presented in this chapter established the foundation for implementing an affordable, accessible, and real-time indoor navigation assistance system for visually impaired individuals. In the next chapter, the implementation details of the system, development environment, experimental setup, and system evaluation results will be presented.

Chapter 6 - Implementation

This chapter presents the implementation of the Smart Vision Assist system. It describes the development environment, system architecture, mobile application implementation, backend processing server, artificial intelligence integration, and external services used throughout the project. The implementation phase focused on developing a real-time assistive navigation solution capable of detecting indoor objects, estimating distances, assessing environmental risks, and providing guidance to visually impaired users. The system was designed using a modular client-server architecture to ensure scalability, maintainability, and efficient processing of computer vision tasks.

6.1. Development Environment

The Smart Vision Assist system was developed using a combination of mobile application technologies, artificial intelligence frameworks, and backend services. Android Studio and Kotlin were used to implement the mobile application, while Python and FastAPI were used to develop the backend processing server. Object detection functionality was implemented using YOLOv11 models [19], while depth estimation was performed using the MiDaS model. Firebase Authentication was integrated to provide secure user registration and login functionality. Additional libraries such as OpenCV and OkHttp were utilized to support image processing and communication between the mobile application and the server.

Development Tools and Technologies:

Table 0-1-6.1 Tools And Technologies

Component	Technology
Mobile Application	Android Studio, Kotlin
Authentication Service	Firebase Authentication
Backend Server	FastAPI (Python)
AI Integration	YOLOv11, MiDaS, PyTorch
Camera Processing	CameraX
Communication	OkHttp
Image Processing	OpenCV

6.2. System Architecture Overview

The Smart Vision Assist system follows a client-server architecture that separates mobile application functionality from artificial intelligence processing. The main components of the system are:

- **Authentication Module:** Handles user registration, login, session management, and secure access using Firebase Authentication.
- **Mobile Application Module:** Developed using Android Studio and Kotlin. It provides the user interface, camera access, visualization of detection results, and feedback delivery.
- **Camera Processing Module:** Uses CameraX to capture real-time image frames and prepare them for transmission to the backend server.
- **Communication Module:** Responsible for transmitting captured frames to the server and receiving processing results using HTTP requests through the OkHttp library.
- **Object Detection Module:** Utilizes two YOLOv11 models to detect common indoor obstacles such as people, chairs, and tables, in addition to navigation-related objects such as doors and stairs.
- **Depth Estimation Module:** Uses the pre-trained MiDaS model to estimate relative depth information from captured images and determine object proximity.
- **Navigation Module:** Analyzes detected objects, their positions, and estimated distances to generate navigation guidance and movement instructions.

- **Risk Assessment Module:** Evaluates environmental hazards based on object type, distance, position, and confidence level, then assigns an appropriate risk level.
- **Feedback Module:** Provides users with navigation instructions through visual overlays, speech feedback using Text-to-Speech, and vibration alerts for potentially dangerous situations.

6.3 Frontend Implementation

The frontend of the Smart Vision Assist system was developed using **Android Studio and Kotlin**. The mobile application serves as the primary interface between the user and the system, allowing users to authenticate, access the camera, receive navigation guidance, and interact with the system in real time. The frontend was designed with simplicity and accessibility in mind to ensure an intuitive user experience while supporting the requirements of visually impaired users.

User Interface Design

The user interface was designed to provide a clean and straightforward navigation experience. Consistent colors, typography, and layout structures were used throughout the application to maintain visual uniformity and improve usability.

The application interface consists of several screens including the **welcome screen, registration screen, login screen, main interface, and camera interface**. Each screen was designed with a specific purpose while maintaining a consistent visual identity across the application.

Authentication Management

User authentication was implemented using **Firebase Authentication**. This service was integrated to provide secure user registration, login, and session management functionality.

When a new user registers, the provided credentials are validated and securely stored using Firebase services. Existing users can log in using their registered email and password. Session persistence was also implemented to improve usability by allowing authenticated users to access the application without repeated login operations.

Core Screens Implemented

The following major screens were implemented within the mobile application:

- **Welcome Screen:** Introduces users to the Smart Vision Assist platform and provides navigation options for account creation and login.
- **Registration Screen:** Allows new users to create accounts using Firebase Authentication.
- **Login Screen:** Authenticates existing users and grants access to the application.
- **Main Screen:** Serves as the central navigation interface and allows users to start environmental scanning.
- **Camera Screen:** Captures real-time image frames and displays detection results returned by the backend server.

6.4. Backend Implementation

The backend server represents the core processing component of the Smart Vision Assist system. It is responsible for receiving image frames from the Android application, performing artificial intelligence analysis, generating navigation guidance, and returning results to the user in real time. The backend was developed using **Python** and **FastAPI**, providing a lightweight and efficient environment for integrating computer vision and deep learning models.

FastAPI Server

The backend server was implemented using **FastAPI**, a modern Python framework designed for building high-performance web APIs. FastAPI was selected due to its lightweight architecture, fast response times, and seamless integration with machine learning libraries.

The server receives image frames transmitted by the Android application through HTTP requests. Each frame is processed by the integrated artificial intelligence models before a JSON response containing detection results and navigation guidance is returned to the client application.

Server Responsibilities

The backend server performs several important functions:

- Receiving image frames from the Android application.
- Executing object detection models.
- Performing depth estimation.
- Calculating object positions and distances.
- Generating navigation guidance.
- Assessing environmental risks.
- Returning processing results in JSON format.

Image Processing Pipeline

Once an image frame is received, the server begins a multi-stage processing pipeline. The image is first decoded and prepared for analysis. The processed image is then forwarded to the object detection and depth estimation modules.

The pipeline was designed to ensure efficient processing while maintaining real-time performance.

The processing workflow consists of:

1. Image reception.
2. Image preprocessing.
3. Object detection.
4. Depth estimation.
5. Navigation analysis.
6. Risk assessment.
7. Response generation.

Navigation Logic

A rule-based navigation engine was implemented to transform object detection results into meaningful movement instructions.

For each detected object, the system determines:

- Object category.
- Confidence score.
- Position within the frame.
- Estimated distance.

Objects are categorized according to their location within the image as:

- Left
- Center
- Right

The navigation engine then analyzes these values to determine whether the user should continue forward, adjust direction, or be alerted to a potential hazard.

Risk Assessment

To improve user safety, a risk assessment mechanism was integrated into the server.

The system evaluates detected objects based on:

- Object type.
- Detection confidence.
- Bounding-box area.
- Estimated depth.

Based on these factors, the environment is classified into one of four risk levels:

- Low Risk
- Medium Risk
- High Risk
- Critical Risk

The calculated risk level influences the guidance and alerts returned to the user.

6.5. AI Integration

Artificial intelligence plays a central role in the Smart Vision Assist system by enabling environmental understanding and navigation assistance. The system integrates object detection and depth estimation models to identify surrounding objects, estimate their relative distances, and generate navigation guidance in real time.

The AI component combines two **YOLOv11** object detection models and the **MiDaS** depth estimation model [20] [12]. The outputs of these models are analyzed by the navigation engine to determine potential hazards and provide safe movement instructions.

YOLOv11 Object Detection

- **Purpose:** The primary purpose of the object detection module is to identify objects present in the user's environment and determine their locations within the camera frame.
- **Functionality:** The system utilizes two YOLOv11 models:
 - A pre-trained YOLOv11 model trained on the COCO dataset for detecting common indoor objects such as people, chairs, tables, and benches.
 - A custom-trained YOLOv11 model developed specifically for indoor navigation tasks, capable of detecting doors and stairs.
- **Output:** The output of the object detection module is a list of detected objects with their corresponding confidence scores and locations.

MiDaS Depth Estimation

- **Purpose:** Object detection alone cannot determine how close an object is to the user. Therefore, a depth estimation module was integrated to provide additional spatial information.
- **Functionality:** The system uses the pre-trained **MiDaS** model to generate a depth map from a single RGB image. The model estimates relative depth values for scene elements without requiring specialized depth sensors.
- **Output:** The depth estimation module produces a depth map representing the relative distance of objects within the environment. Objects with higher depth values are generally closer to the user, while lower values indicate greater distance.

Object Position and Distance Analysis

- **Purpose:** The purpose of this stage is to transform raw detection results into meaningful spatial information that can be used for navigation.
- **Functionality:** After object detection and depth estimation, the system calculates
 - Object position within the frame.
 - Bounding-box area.
 - Relative depth value.
 - Estimated distance category.
 - Objects are classified according to their horizontal location as:
 - Left
 - Center
 - Right
- Distance estimation is based on a combination of:
 - Bounding-box area.
 - MiDaS depth values.

Using both measurements improves reliability compared to relying on a single metric.

- **Output:** Each detected object is assigned a position and distance category, which are later used by the navigation engine.

Navigation Decision Generation

- **Purpose:** The navigation module converts AI outputs into movement instructions that assist users during navigation.
- **Functionality:** The navigation engine analyzes
 - Detected objects.
 - Object positions.
 - Estimated distances.
 - Environmental risk level.

A priority-based strategy was implemented to ensure that critical hazards receive immediate attention.

The current priority order is:

1. Stairs
2. Persons
3. Obstacles
4. Doors
5. Safe path

Doors are treated as navigation landmarks rather than obstacles, while stairs receive the highest priority due to their potential danger.

- **Output:** The module generates navigation instructions such as
- Move left
- Move right
- Obstacle ahead
- Door on the left
- Safe to proceed

These instructions are transmitted to the Android application and delivered through visual, audio, and vibration feedback.

6.6. Services and Integrations

The Smart Vision Assist system integrates several third-party services and libraries to support authentication, computer vision processing, camera management, and communication between system components. These services reduced development complexity and provided reliable solutions for key functionalities within the application.

Third-Party Services Overview

Table 0-2-6.6: Third party services

Service	Purpose
Firestore Authentication	User registration, login, and session management
CameraX	Real-time camera access and image acquisition
OkHttp	Communication between the Android application and backend server
FastAPI	Backend API development and request handling
YOLOv11	Object detection and scene understanding
MiDaS	Relative depth estimation
PyTorch	Deep learning model execution

6.7. Challenges Encountered

Throughout the development of Smart Vision Assist, several technical challenges were encountered during both the mobile application and artificial intelligence implementation phases.

- **Real-Time Performance:** Processing image frames continuously while maintaining acceptable response times required careful optimization of image transmission and server-side processing.
- **False Object Detections:** During testing, the object detection model occasionally misclassified environmental elements, particularly in complex indoor scenes. Additional testing and model refinement were necessary to improve detection reliability.
- **Depth Estimation Accuracy:** Since MiDaS provides relative depth rather than exact distance measurements, additional logic was required to combine depth information

with bounding-box area for more reliable distance estimation.

- **Navigation Decision Generation:** Transforming raw detection results into meaningful navigation instructions required multiple iterations of rule-based logic to avoid contradictory or repetitive guidance.
- **Feedback Management:** Continuous speech generation created excessive repetition during testing. This issue was addressed by introducing duplicate-message filtering and controlled feedback updates.
- **Dataset Preparation:** Training the custom YOLOv11 model required collecting and organizing images of doors and stairs under different lighting conditions and viewing angles to improve generalization.

The identified challenges were gradually addressed through iterative testing, model refinement, and continuous adjustments to the navigation logic.

6.8. Results and Observations

This section presents the experimental results obtained from implementing and testing the Smart Vision Assist system. The evaluation includes the training performance of the YOLOv11 object detection model, classification results, quantitative performance metrics, and the functionality of the developed Android application. These results demonstrate the effectiveness of the proposed system in supporting real-time indoor navigation.

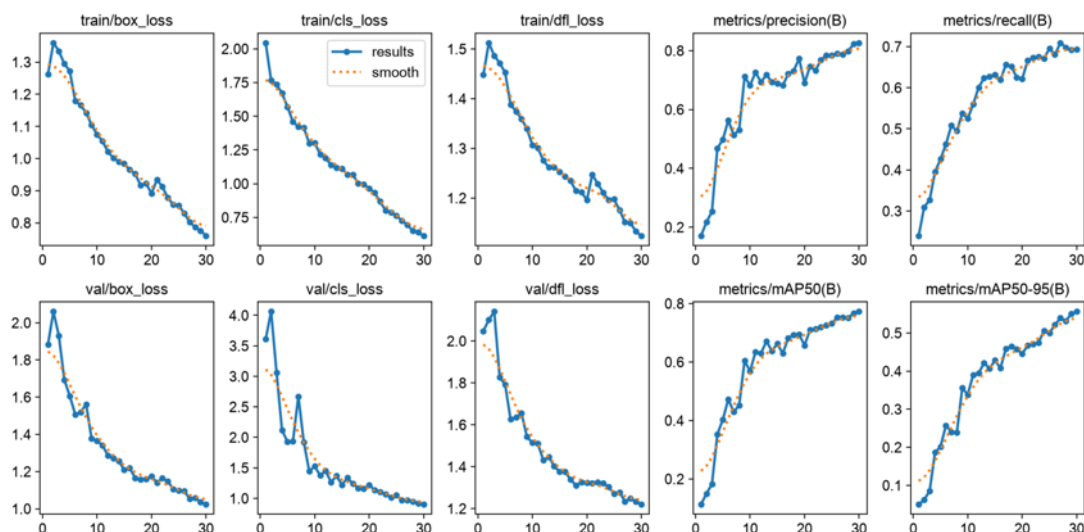


Figure 17-6.8: Training Performance

The training and validation performance of the YOLOv11 model was evaluated over 30 training epochs. The training and validation losses decreased steadily, indicating stable learning and successful model convergence without significant overfitting. Additionally, the Precision, Recall, mAP@0.5, and mAP@0.5:0.95 metrics improved consistently throughout training, demonstrating the model's ability to accurately detect and localize indoor objects. These results indicate that the trained model provides reliable performance for real-time indoor object detection and is suitable for integration into the Smart Vision Assist system.

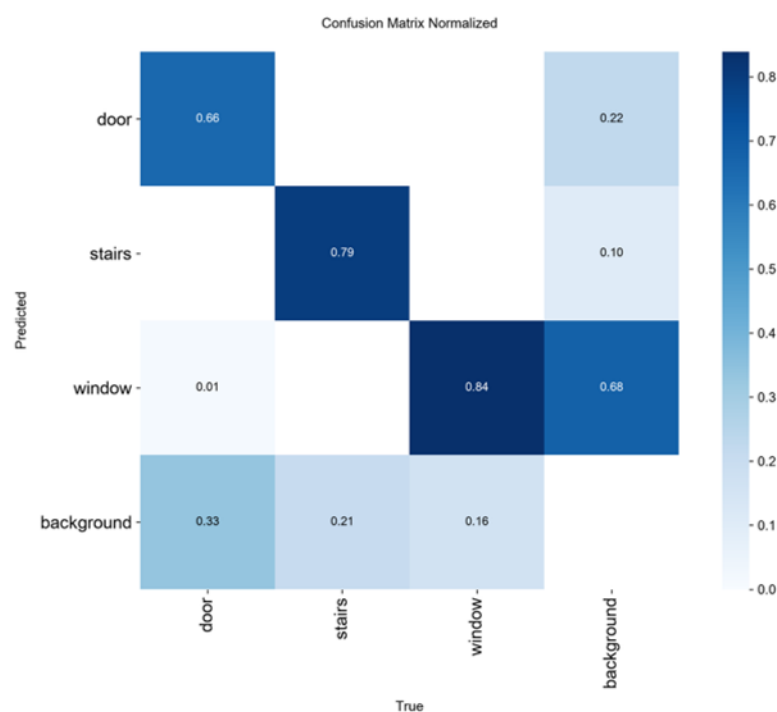


Figure 18-6.8: Confusion Matrix

The confusion matrix was used to evaluate the classification performance of the trained YOLOv11 model. Most predictions were concentrated along the main diagonal, indicating that the model correctly classified the majority of indoor object classes. The limited number of misclassifications demonstrates good discrimination between different objects, confirming that the trained model achieved reliable classification performance for indoor navigation tasks.

Table 3: Performance Metrics

Performance Metric	Result
Precision	82.64%
Recall	69.35%
mAP@0.5	77.26%
mAP@0.5:0.95	55.62%

The final performance metrics demonstrate that the trained YOLOv11 model achieved reliable object detection performance. The obtained Precision, Recall, mAP@0.5, and mAP@0.5:0.95 values indicate accurate object classification and localization, making the model suitable for real-time indoor navigation within the Smart Vision Assist system.

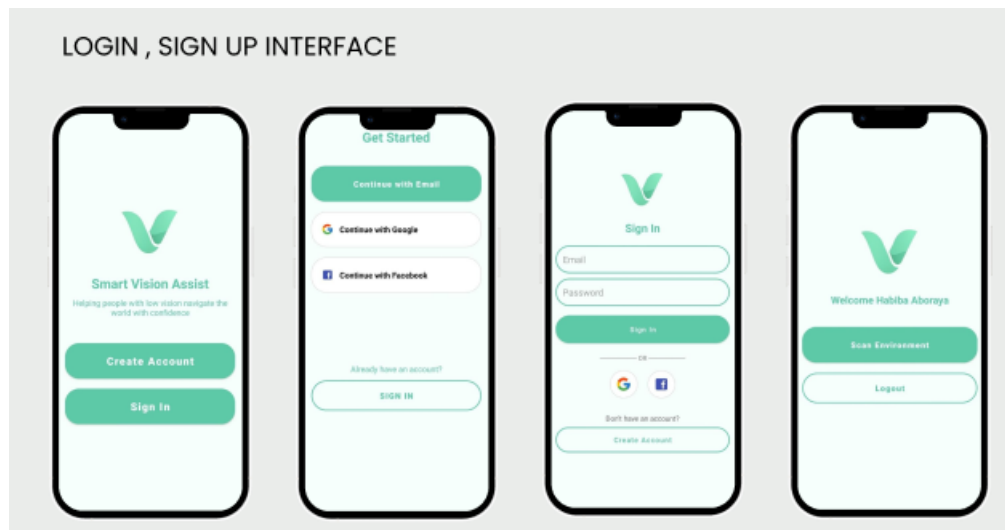


Figure 19-6.8: Login, Signup Interface

The authentication module provides users with secure access to the application through Firebase Authentication. New users can create an account using their email address, while existing users can log in using their registered credentials. After successful authentication, the application automatically redirects the user to the main interface where indoor navigation can be initiated.

The home screen displays a personalized welcome message together with the Scan Environment button, which launches the camera interface and starts the real-time navigation process. This simple interface minimizes user interaction and allows quick access to the application's main functionality.

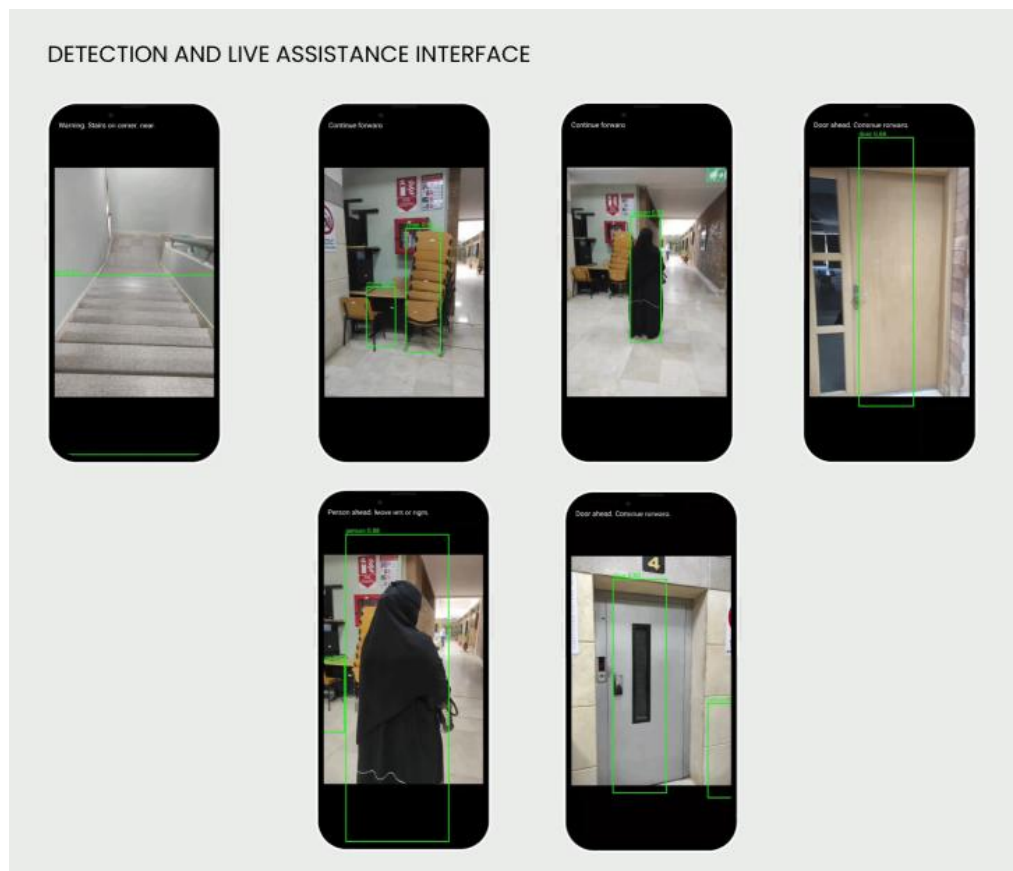


Figure 20-6.8: Detection And Assistance Interface

The camera interface continuously captures live video frames and transmits them to the backend server for processing. Each frame is analyzed using the YOLOv11 object detection model and the MiDaS depth estimation model. The processed results are then returned to the Android application, where detected objects are displayed using bounding boxes together with the corresponding navigation guidance.

The presented results demonstrate successful detection of various indoor objects, including stairs, chairs, doors, people, and elevators. Based on the detected object type, its position within the frame, and the estimated relative depth, the system generates navigation instructions to assist users in safely navigating indoor environments.

Voice guidance generated using Android Text-to-Speech and vibration alerts are simultaneously provided whenever necessary, allowing visually impaired users to receive immediate feedback without relying solely on visual information.

6.9. Summary

This chapter presented the implementation of the **Smart Vision Assist system**. The development process involved the creation of an Android application, a FastAPI-based backend server [15], and the integration of artificial intelligence models for object detection and depth estimation.

The frontend implementation focused on user authentication, camera integration, detection visualization, and feedback delivery, while the backend server handled image processing, navigation analysis, and risk assessment. Artificial intelligence capabilities were achieved through the integration of YOLOv11 object detection models and the MiDaS depth estimation model, enabling the system to identify indoor objects, estimate relative distances, and generate navigation guidance in real time.

In addition, several third-party services and libraries were incorporated to support authentication, communication, and computer vision processing. Various challenges encountered during development were addressed through iterative testing and refinement, resulting in a functional prototype capable of assisting users in understanding and navigating indoor environments.

The implementation outcomes demonstrate the effectiveness of the proposed approach and provide a solid foundation for future enhancements and expanded navigation capabilities

Chapter 7 - Testing

7.1 Introduction

Testing is a fundamental phase in the software development lifecycle as it ensures that the developed system satisfies its specified requirements and performs reliably under different operating conditions. Since Smart Vision Assist is intended to assist visually impaired users during indoor navigation, the system must provide accurate object detection, reliable distance estimation, and timely navigation instructions [21].

This chapter presents the testing strategies, methodologies, tools, and environments used to evaluate the Smart Vision Assist system. Both functional and non-functional testing were conducted to verify the correctness, reliability, usability, and performance of the application. The testing process focused on validating the Android application, backend services, Artificial Intelligence models, and navigation assistance mechanisms to ensure that the proposed system can effectively support users in understanding and navigating indoor environments.

7.2 Testing Strategies

7.2.1 Unit Testing

Purpose:

To verify the correctness of individual software components and functions independently.

Tools:

PyTest, Android Studio Emulator, Manual Testing.

Approach:

Each software module was tested separately, including object detection, depth estimation, risk assessment, speech generation, and vibration mechanisms. Testing individual components allowed early identification and correction of implementation issues.

7.2.2 Integration Testing

Purpose:

To evaluate the interactions between different system modules.

Tools:

Postman, Android Emulator, FastAPI Testing Tools.

Approach:

The integration between the Android application, FastAPI backend server, YOLOv11 object detection models, MiDaS depth estimation model, and navigation engine was tested to ensure that all components communicate correctly and exchange information without errors.

7.2.3 System Testing

Purpose:

To evaluate the complete Smart Vision Assist system as an integrated application.

Tools:

Android Emulator, Physical Android Devices, Manual Testing.

Approach:

End-to-end testing scenarios were executed to validate the entire workflow, beginning with image acquisition and ending with navigation guidance generation through speech and vibration notifications.

7.2.4 Performance Testing

Purpose:

To assess the responsiveness and efficiency of the system under continuous operation.

Tools:

Android Profiler, FastAPI Logging Tools, Manual Stress Testing.

Approach:

The system was tested by continuously processing video streams and camera frames to measure response times, frame processing speed, and the stability of real-time navigation guidance.

7.2.5 Usability Testing

Purpose:

To evaluate the ease of use and effectiveness of the application interface.

Tools:

Manual User Testing and Questionnaire-Based Evaluation.

Approach:

Users interacted with the application and evaluated the clarity of navigation instructions, response times, and overall ease of use. Feedback was used to improve the user experience and simplify application interactions.

7.2.6 Reliability Testing

Purpose:

To ensure that the application operates consistently without crashes or failures.

Tools:

Android Studio Logcat, Emulator Testing, Physical Device Testing.

Approach:

The application was executed continuously over extended periods while processing different indoor scenarios. The system was monitored to identify memory leaks, crashes, and unexpected behavior.

7.3 Testing Tools and Environment

1. **Android Studio Emulator**

- Used for testing application functionality on different Android versions and screen sizes.

2. **Physical Android Devices**

- Used to validate real-world application behavior and performance.

3. **PyTest**

- Used for testing Python backend functions and navigation logic.

4. **Postman**

- Used for testing FastAPI endpoints and verifying communication between the mobile application and backend services.

5. **FastAPI Logging Tools**

- Used to monitor request processing and identify system errors.

6. **Android Profiler**

- Used to measure memory consumption, CPU utilization, and application performance.

7. **Logcat**

- Used for debugging and identifying runtime exceptions and application failures.

7.4 Test Cases and Results

7.4.1 Unit Testing

Test Case 1: Verify YOLOv11 Object Detection

Description:

Ensure that the object detection module correctly identifies indoor objects.

Expected Result:

The system detects indoor objects and returns their labels, confidence scores, and locations.

Actual Result:

Passed.

Test Case 2: Verify MiDaS Depth Estimation

Description:

Ensure that the depth estimation module generates valid depth maps.

Expected Result:

The system produces relative depth values for detected objects.

Actual Result:

Passed.

7.4.2 Integration Testing

Test Case 3: Verify Camera-to-Backend Communication

Description:

Ensure that captured images are successfully transmitted to the backend server.

Expected Result:

Frames are received and processed correctly by the backend.

Actual Result:

Passed.

Test Case 4: Verify AI Integration

Description:

Ensure that object detection and depth estimation outputs are successfully integrated within the navigation engine.

Expected Result:

Detected objects are assigned distance information and risk levels.

Actual Result:

Passed.

7.4.3 System Testing

Test Case 5: End-to-End Navigation Assistance

Description:

Test the complete process from image capture to navigation guidance generation.

Expected Result:

The system generates appropriate movement instructions and feedback notifications.

Actual Result:

Passed.

7.4.4 Performance Testing

Test Case 6: Continuous Video Processing

Description:

Evaluate system performance while continuously processing video streams.

Expected Result:

The application maintains acceptable response times and stable performance.

Actual Result:

Passed with minor delays during complex scenes containing multiple objects.

7.4.5 Usability Testing

Test Case 7: Navigation Guidance Evaluation

Description:

Evaluate the clarity and usefulness of navigation instructions.

Expected Result:

Users can easily understand and follow navigation guidance.

Actual Result:

Passed.

7.4.6 Reliability Testing

Test Case 8: Long-Duration Application Execution

Description:

Run the application continuously for extended periods.

Expected Result:

The application remains stable without crashes or significant memory issues.

Actual Result:

Passed.

7.5 Summary

This chapter presented the testing procedures applied to the Smart Vision Assist system to ensure its quality, reliability, and effectiveness. Various testing strategies were employed, including unit testing, integration testing, system testing, performance testing, usability testing, and reliability testing.

Several testing tools and environments were utilized, including Android Studio Emulator, Physical Android Devices, PyTest, Postman, Android Profiler, FastAPI logging tools, and Logcat. The testing process verified the correctness of object detection, depth estimation, backend communication, navigation logic, and user feedback mechanisms.

The results demonstrated that the Smart Vision Assist system successfully performs real-time object detection and navigation assistance while maintaining acceptable responsiveness and reliability. Although minor performance degradation was observed in highly complex indoor scenes, the overall results indicate that the proposed system is capable of assisting visually impaired users in understanding and navigating indoor environments effectively.

In the next chapter, we will present the conclusions and future work of the Smart Vision Assist system. The chapter will summarize the project's main achievements, discuss its limitations, and propose future enhancements that can further improve navigation accuracy, environmental understanding, and overall user experience.

Chapter 8 - Conclusion and Future Work

8.1 Conclusion

The development of Smart Vision Assist demonstrates the potential of integrating Artificial Intelligence and mobile technologies to improve accessibility and independent navigation for visually impaired individuals. By combining real-time object detection, monocular depth estimation, and multimodal feedback mechanisms, the proposed system provides users with environmental awareness and navigation assistance using only a smartphone.

The project successfully implemented a complete prototype consisting of an Android application, a FastAPI backend server, and Artificial Intelligence models based on YOLOv11 and MiDaS. The system is capable of detecting important indoor objects, estimating their relative distances, assessing environmental risks, and generating navigation instructions through audio and vibration feedback.

The results obtained during development and testing demonstrate the feasibility of using deep learning and computer vision techniques to support visually impaired users in understanding and navigating indoor environments. Furthermore, the project highlights the importance of designing affordable and accessible assistive technologies that rely on commonly available devices rather than specialized and expensive hardware.

Although the current implementation has certain limitations regarding complex scenes and depth estimation accuracy, it establishes a solid foundation for future improvements and demonstrates the significant role that Artificial Intelligence can play in enhancing accessibility and promoting independent living for people with visual impairments.

8.2 Future Work

Developing Smart Vision Assist opens several opportunities for future research and enhancements. By extending the current capabilities of the system, the project can evolve into a comprehensive assistive platform that provides more intelligent and reliable navigation support for visually impaired users.

1. Research and Pilot Studies

- Conduct pilot studies with visually impaired individuals to evaluate system usability and effectiveness in real-world environments.
- Gather continuous feedback from users and accessibility experts to improve navigation guidance and overall user experience.
- Evaluate system performance across different indoor environments and lighting conditions.

2. Enhanced Computer Vision Capabilities

- Expand the object detection dataset to include additional indoor and outdoor objects.
- Improve model generalization by training on larger and more diverse datasets.
- Introduce semantic scene understanding to better recognize environmental contexts and relationships between objects.

3. Improved Navigation Intelligence

- Implement a Safe Corridor Detection module to identify obstacle-free walking paths.
- Develop path planning algorithms that generate more intelligent movement recommendations.
- Incorporate context-aware navigation strategies that adapt instructions according to environmental complexity.

4. Enhanced Distance Estimation

- Integrate advanced depth estimation techniques to improve distance accuracy.
- Investigate the use of sensor fusion approaches by combining camera information with smartphone sensors.
- Improve risk assessment models by considering object motion and user movement.

5. Performance Optimization

- Optimize Artificial Intelligence models for real-time mobile execution.
- Reduce processing latency and improve frame processing speed.
- Implement lightweight model versions suitable for low-end mobile devices.

6. Accessibility Enhancements

- Support multiple languages for speech feedback generation.
- Introduce customizable vibration patterns and audio notifications.
- Add voice command functionality to allow hands-free interaction with the application.

8.2.1 Impact on Visually Impaired Individuals

The implementation of Smart Vision Assist can significantly improve the quality of life of visually impaired users by providing greater environmental awareness and navigation support.

Enhanced Independence:

Users can navigate indoor environments more confidently and perform daily activities with reduced reliance on assistance from others.

Improved Safety:

Real-time detection of obstacles and hazards helps users avoid collisions and navigate more safely.

Increased Accessibility:

The system provides an affordable assistive solution by utilizing standard smartphone hardware without requiring specialized equipment.

8.2.2 Benefits to Society and Institutions

The proposed system also provides benefits beyond individual users.

Accessibility Inclusion:

The system contributes to the development of inclusive technologies that support equal access to public and private environments.

Educational and Research Opportunities:

The project provides a practical framework for future research in computer vision, assistive technologies, and human-centered Artificial Intelligence applications.

Healthcare and Rehabilitation Support:

The application can complement rehabilitation programs and assist healthcare professionals in providing technological support for visually impaired individuals.

8.2.3 Challenges and Considerations

Despite the promising results, several challenges remain.

Environmental Variations:

Complex indoor scenes and poor lighting conditions can affect detection accuracy.

Depth Estimation Limitations:

Monocular depth estimation provides relative rather than exact distances.

Real-Time Processing Constraints:

Maintaining high performance while processing continuous video streams remains computationally demanding.

User Adaptation:

Different users may require personalized feedback mechanisms and navigation preferences.

8.2.4 Community Involvement and Collaboration

Successful deployment of assistive technologies requires collaboration among different stakeholders.

Academic Collaboration:

Cooperation with universities and research institutions can support further development and evaluation.

Accessibility Organizations:

Partnerships with organizations serving visually impaired individuals can provide valuable user feedback and testing opportunities.

Open-Source Contributions:

Encouraging community contributions can accelerate system improvements and increase accessibility innovation.

8.2.5 Potential for Expansion Beyond Indoor Navigation

The technologies developed in this project have applications beyond indoor navigation assistance.

Outdoor Navigation:

The system can be expanded to support outdoor environments and urban navigation.

Smart Glasses Integration:

Future versions may integrate with wearable devices and smart glasses for hands-free assistance.

Emergency Assistance:

The application can be extended to detect emergency situations and automatically notify caregivers or emergency services.

Robotics and Smart Environments:

The navigation and scene understanding techniques developed in this project can also support autonomous robots and intelligent building systems.

8.2.6 Continuous Improvement and Innovation

The development of Smart Vision Assist does not end with the current implementation.

Continuous feedback collection, iterative model refinement, and adoption of emerging Artificial Intelligence technologies will allow the system to evolve and remain effective in meeting user needs. Future technological advances in computer vision, mobile computing, and accessibility solutions can further enhance the capabilities and reliability of the system.

8.2.7 Vision for the Future

The ultimate vision of Smart Vision Assist is to provide an intelligent, affordable, and universally accessible navigation assistant that empowers visually impaired individuals to move safely and independently.

By combining Artificial Intelligence, mobile technologies, and human-centered design principles, the project aims to contribute toward a future where technology serves as an effective tool for inclusion, accessibility, and independent living for people with visual impairments worldwide.

References

- [1] IEEE, "IEEE Standard for System and Software Life Cycle Processes, IEEE Std 12207-2017," IEEE , NY , 2017.
- [2] IEEE, "IEEE Recommended Practice for Software Requirements Specifications—IEEE Std 830-1998," IEEE, NY, 1998.
- [3] International Organization for Standardization (ISO), "ISO 40500:2012, Information Technology—W3C Web Content Accessibility Guidelines (WCAG) 2.0,," ISO, Geneva, 2012.
- [4] R. S. ., F. E. S. Bineeth Kuriakose, "DeepNAVI: A deep learning based smartphone navigation assistant for people with visual impairments," February 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417422017432>. [Accessed March 2026].
- [5] [Online]. Available: https://www.researchgate.net/figure/Example-of-the-typical-layered-structure-of-a-CNN-18_fig2_337089553.
- [6] M. H. R. H. Rahima Khanam, "A comprehensive review of convolutional neural networks for defect detection in industrial applications.," *IEEE Access*, 2024.
- [7] G. M. P. Anandh Nagarajan, "Hybrid Optimization-Enabled Deep Learning for Indoor Object Detection and Distance Estimation to Assist Visually Impaired Persons," February 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0965997822002630>. [Accessed February 2026].
- [8] [Online]. Available: <https://link.springer.com/article/10.1007/s10462-025-11284-w>.
- [9] R. Khanam, "YOLOv11: An Overview of the Key Architectural Enhancements," October 2024. [Online]. Available: <https://arxiv.org/html/2410.17725v1>. [Accessed March 2026].
- [10] [Online]. Available: <https://www.nature.com/articles/s41598-025-24850-7>.
- [11] [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0925231220320014>.
- [12] N. Khan, "Getting Started with Depth Estimation using MiDaS and Python," june 2023. [Online]. Available: <https://medium.com/@nbeel.original/getting-started-with-depth-estimation-using-midas-and-python-d0119bfe1159>. [Accessed March 2026].
- [13] S. S. ., A. S. Shashwat Singh, "Android Application Development using Android Studio and PHP Framework," 2016. [Online]. Available: <https://www.semanticscholar.org/paper/Android-Application-Development-using-Android-and-Singh-Sharma/07e84508aaca29a9bb22632dd67b9cc203a77e14>. [Accessed November 2025].
- [14] [Online]. Available: <https://medium.com/@ruwanyahettiarachchi/getting-started-with-android-studio-installation-and-first-project-0e186a689cc6>.
- [15] P. R.Saraf, "A Review on Firebase (Backend as A Service) for Mobile Application Development," 2022. [Online]. Available: https://www.researchgate.net/publication/358244355_A_Review_on_Firebase_Backend_as_A_Service_for_Mobile_Application_Development. [Accessed January 2026].

- [16] [Online]. Available: <https://medium.com/@eeswarez/speech2text-thru-ai-ml-part-i-4472b5a3a2f7>.
- [17] [Online]. Available: <https://www.letsenvision.com/blog/introducing-the-redesigned-envision-app>.
- [18] [Online]. Available: <https://www.bemyeyes.com/>.
- [19] "YOLO model comparison," January 2026. [Online]. Available: <https://apmonitor.com/dde/index.php/Main/PoseEstimation>.
- [20] X. Cheng, "Object Detection service with YOLO and FastAPI," September 2022. [Online]. Available: <https://billtcheng2013.medium.com/object-detection-service-with-yolo-and-fastapi-af1318ee73ed>. [Accessed April 2026].
- [21] IEEE, "IEEE Standard for Software and System Test Documentation—IEEE Std 829-2008," IEEE, NY, 2008.
- [22] C. Devanshi, june 2025. [Online]. Available: <https://www.ijssat.org/papers/2025/2/3924.pdf>.
- [23] V. H. a. R. B. Milan Sonka, "Image processing, analysis and machine vision," *Springer*, 2013.
- [24] K. C. Z. S. Y. G. Zhengxia Zou, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, 2023.
- [25] I. B. Hassani, "comparative analysis of YOLO algorithm," February 2026. [Online]. Available: [400327771_From_YOLO_V1_to_YOLO_V11_comparative_analysis_of_YOLO_a_lgorithm_review](https://www.researchgate.net/publication/400327771_From_YOLO_V1_to_YOLO_V11_comparative_analysis_of_YOLO_algorithm_review). [Accessed March 2026].
- [26] X. Y. Jafar Saniie, "Visual Impairment Spatial Awareness System for Indoor Navigation and Daily Activities," 2025. [Online]. Available: <https://www.mdpi.com/2313-433X/11/1/9>. [Accessed February 2026].
- [27] [Online]. Available: https://www.researchgate.net/figure/Flowchart-of-the-Computer-Vision-Process-This-diagram-illustrates-the-sequential-stages_fig3_377143161.
- [28] [Online]. Available: https://www.researchgate.net/figure/Convolutional-Neural-Network-CNN10_fig2_399651691.
- [29] [Online]. Available: https://www.researchgate.net/figure/Convolutional-Neural-Network-CNN10_fig2_399651691.
- [30] [Online]. Available: <https://link.springer.com/article/10.1007/s10462-025-11284-w>.

Appendix A - Software Requirements Specifications IEEE 830

Software Requirements Specification

for

«Smart Vision Assist»

Version «0.1»

Date : June 28 , 2026

Prepared by « Aya Abdelhakam

Khadija Mohamed

Hana Khamis

Habiba Aboraya »

«Arab academy for science and technology»

Introduction

Purpose

This document specifies the software requirements for the Smart Vision Assist system, an Artificial Intelligence-based mobile application designed to assist visually impaired individuals during indoor navigation.

The system utilizes Computer Vision and Deep Learning techniques to detect surrounding obstacles, estimate their relative distances, assess potential risks, and provide immediate warning notifications through speech and vibration feedback.

The purpose of this document is to define the functional and non-functional requirements of the proposed system and provide a clear specification for development, testing, and future maintenance activities.

Document Conventions

This specification follows standard documentation conventions.

- Section headings are numbered hierarchically.
- Functional requirements are identified using unique identifiers (REQ-x.x).
- Non-functional requirements are identified using unique identifiers (NF-x.x).
- Important terms and module names are written in bold.
- Figures, tables, and diagrams are numbered according to chapter conventions.

Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

Developers

To understand the system architecture, requirements, and implementation details.

Project Supervisors

To evaluate project objectives and monitor development progress.

Testers

To derive test cases and validate system functionality.

Documentation Writers

To prepare user manuals and technical documentation.

End Users

To understand the capabilities and limitations of the proposed system.

The document is organized as follows:

- Chapter 1 – Introduction
- Chapter 2 – Overall Description
- Chapter 3 – System Features
- Chapter 4 – External Interface Requirements
- Chapter 5 – System Modules
- Chapter 6 – Nonfunctional Requirements
- Chapter 7 – Testing
- Chapter 8 – Conclusion and Future Enhancements

Product Scope

Smart Vision Assist is a mobile assistive application developed to improve indoor navigation for visually impaired individuals.

The system aims to:

- Improve independent mobility.
- Increase user awareness of surrounding obstacles.
- Reduce navigation risks and collision incidents.

- Provide real-time environmental understanding.
- Deliver multimodal feedback through speech and vibration notifications.
- Operate using ordinary smartphone cameras without requiring specialized hardware.

The system combines Artificial Intelligence, Computer Vision, and mobile technologies to create a low-cost and portable assistive solution.

Overall Description

Product Perspective

Smart Vision Assist is a standalone mobile assistive application that integrates Computer Vision, Artificial Intelligence, and mobile technologies to support visually impaired users during indoor navigation.

The system continuously captures live video streams from the smartphone camera and processes them using object detection and monocular depth estimation models to understand the surrounding environment and identify potential hazards.

Product Features

The major features of the system include:

- Real-time object detection.
- Indoor obstacle recognition.
- Relative distance estimation.
- Risk assessment and hazard evaluation.
- Speech notifications.
- Vibration notifications.
- Real-time video processing.
- User-friendly mobile interface.

User Classes and Characteristics

- Visually Impaired Users

Primary users of the application who depend on speech and vibration notifications for navigation assistance.

- Caregivers
- Secondary users

who may assist in application installation and configuration.

- System Administrators and Developers

Responsible for system maintenance, updates, and performance improvements.

Operating Environment

- Hardware Platform

Android smartphones equipped with cameras and vibration motors.

- Operating System

Android 10 or later.

- Software Components
 - Android Studio
 - CameraX
 - FastAPI
 - YOLO11
 - MiDaS
 - Text-to-Speech Engine
 - Android Vibration Service

Design and Implementation Constraints

- Limited computational resources of mobile devices.
- Real-time processing requirements.
- Dependence on smartphone camera quality.
- Performance variations under poor lighting conditions.
- Memory and battery limitations.
- Compatibility requirements across Android devices.

User Documentation

The system documentation includes:

- User Manual
- Installation Guide
- Quick Start Guide
- Troubleshooting Guide
- Technical Documentation

Assumptions and Dependencies

- Users possess Android smartphones.
- Smartphone cameras function correctly.
- Text-to-Speech services are available.
- Vibration services are supported by the device.
- Artificial Intelligence models are properly loaded and configured.

System Features

Feature: Object Detection

Brief Description:

The system detects indoor obstacles such as chairs, tables, doors, stairs, walls, windows, and people in real time using the YOLO11 model.

Feature: Depth Estimation

Brief Description:

The system estimates the relative distances between the user and detected objects using MiDaS monocular depth estimation.

Feature: Risk Assessment

Brief Description:

The system evaluates detected objects and their estimated distances to determine potential hazards.

Feature: Speech Notification

Brief Description:

The system generates verbal warning messages describing detected obstacles and risk levels.

Feature: Vibration Notification

Brief Description:

The system provides tactile warning feedback through vibration patterns whenever dangerous obstacles are detected.

External Interface Requirements

User Interface Overview

The application provides a simple and accessible graphical user interface that enables users to:

- Launch the application.
- Activate camera processing.
- Receive voice notifications.
- Receive vibration notifications.
- View system status.

Hardware Interfaces

The system interfaces with:

- Smartphone Camera

- Speaker
- Vibration Motor
- Smartphone Processor
- Device Memory

Software Interfaces

The system integrates with:

- Android Operating System
- CameraX API
- FastAPI Backend
- YOLO11 Model
- MiDaS Model
- Android Text-to-Speech Engine
- Android Vibration Service

System Modules

Object Detection Module

Description and Priority

This module detects surrounding indoor objects using the YOLO11 model.

Priority: High

Stimulus/Response Sequence

Stimulus:

Camera captures a frame.

Response:

System identifies objects and returns their labels and locations.

Functional Requirements

REQ-1.1: The system shall capture frames continuously from the smartphone camera.

REQ-1.2: The system shall identify multiple objects within a single frame.

REQ-1.3: The system shall display object labels and bounding boxes.

Depth Estimation Module

Description and Priority

This module estimates the relative distance of detected obstacles.

Priority: High

Stimulus/Response Sequence

Stimulus:

Detected objects are received from the object detection module.

Response:

Relative depth values are generated.

Functional Requirements

REQ-2.1: The system shall estimate relative object distances.

REQ-2.2: The system shall generate depth maps from camera frames.

REQ-2.3: The system shall associate depth values with detected objects.

Risk Assessment Module

Description and Priority

This module determines whether detected objects represent potential dangers.

Priority: High

Stimulus/Response Sequence

Stimulus:

Object classes and depth information are received.

Response:

Risk levels are calculated.

Functional Requirements

REQ-3.1: The system shall evaluate object proximity.

REQ-3.2: The system shall assign risk levels.

REQ-3.3: The system shall trigger warning mechanisms when hazards are detected.

Notification Module

Description and Priority

This module generates speech and vibration warnings.

Priority: High

Stimulus/Response Sequence

Stimulus:

Risk assessment identifies dangerous obstacles.

Response:

Speech and vibration notifications are generated.

Functional Requirements

REQ-4.1: The system shall generate spoken warnings.

REQ-4.2: The system shall activate vibration feedback.

REQ-4.3: The system shall provide notifications in real time.

Nonfunctional Requirements

Performance

NF-1.1: The system shall process video frames in real time.

NF-1.2: Notification latency shall not exceed two seconds.

Security

NF-2.1: The application shall protect locally stored data.

NF-2.2: The application shall prevent unauthorized access to configuration settings.

Usability

NF-3.1: The user interface shall be simple and accessible.

NF-3.2: Users shall be able to operate the application with minimal interaction.

Reliability

NF-4.1: The application shall operate continuously without unexpected failures.

NF-4.2: The application shall recover gracefully from runtime exceptions.

Scalability

NF-5.1: The system shall support future integration of additional AI models and navigation features.

Compatibility

NF-6.1: The application shall operate on Android devices running Android 10 or later.

NF-6.2: The application shall support different smartphone screen sizes and hardware configurations.

Testing

Testing Strategies

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing
- User Acceptance Testing

Testing Results

Testing activities verified:

- Correct object detection.
- Accurate relative depth estimation.
- Proper risk assessment.
- Correct speech generation.
- Correct vibration notifications.
- Stable system performance.

Conclusion and Future Enhancements

Summary

Smart Vision Assist provides an Artificial Intelligence-based assistive solution that improves indoor navigation for visually impaired users through real-time object detection, depth estimation, and multimodal warning notifications.

Future Enhancements

- Outdoor navigation support.
- GPS integration.
- Object tracking capabilities.
- Personalized warning configurations.
- Wearable device integration.
- Cloud-based model updates.
- Multi-language support.

Appendix B - Software Test Plan

IEEE 829

Software Test Plan: Smart Vision Assist

Document ID: SVA-TP-2026-v1.0

1. Test Plan Identifier

Test Plan ID: SVA-TP-2026-v1.0

2. References

- Software Requirements Specification (SRS) Document
- IEEE 829 Standard for Software Test Documentation [21]
- IEEE 830 Standard for Software Requirements Specification [2]
- System Design Document and UML Diagrams
- Functional Requirements and Use Case Specifications
- User Interface Design Document
- Manual Testing Document and Test Case Table

3. Introduction

This document defines the Software Test Plan for Smart Vision Assist , an Artificial Intelligence-based mobile application designed to assist visually impaired individuals during indoor navigation. The system utilizes Computer Vision and Deep Learning technologies to detect surrounding obstacles, estimate relative distances, assess potential risks, and provide real-time speech and vibration notifications.

The purpose of this test plan is to ensure that the application satisfies all functional and non-functional requirements and delivers reliable, accurate, secure, and real-time assistance to visually impaired users.

4. Test Items

The following components of the system will be tested:

- Android Application Interface
- Camera Module and Image Acquisition
- FastAPI Backend Communication
- YOLO11 Object Detection Module
- MiDaS Depth Estimation Module
- Navigation Engine
- Risk Assessment Module
- Text-to-Speech (TTS) Notification System
- Vibration Feedback System
- Settings and Preferences Module
- Error Handling and Recovery Mechanisms
- Performance and Real-Time Processing

5. Software Risk Issues

- Reduced performance on low-end mobile devices
- Inaccurate object detection under poor lighting conditions
- Delays in image transmission and processing
- Failure of speech notifications
- Failure of vibration notifications
- Incorrect depth estimation for certain obstacles
- Network connectivity issues between the application and backend server
- Excessive memory or CPU utilization during real-time processing
- Unexpected application crashes during prolonged usage
- Device compatibility issues across different Android versions

6. Features to be Tested

- FR-001 to FR-030 (All Functional Requirements listed in the SRS)

The following functionalities will be tested:

- Camera initialization and frame capture
- Real-time object detection and classification
- Relative distance estimation
- Risk assessment and warning generation
- Speech notifications and vibration alerts
- Environmental awareness and navigation assistance
- User settings and preferences
- Error messages and recovery procedures
- User interface responsiveness and accessibility
- Communication between Android application and backend server
- System behavior under different environmental conditions

7. Features Not to be Tested

- Internal implementation details of YOLO11 and MiDaS models
- AI model training procedures and training datasets
- FastAPI source code implementation details
- Internal functionality of third-party Android libraries
- Smartphone operating system services not directly related to the application
- Hardware-level implementation of Android Text-to-Speech engine

8. Approach

Testing activities will involve a combination of manual and black-box testing techniques.

Testing phases include:

- Unit Testing for individual system modules
- Integration Testing for communication between system components
- System Testing for complete application functionality
- Black-Box Testing for user-facing functionalities
- Exploratory Testing for real-world navigation scenarios
- Regression Testing after bug fixes and modifications
- Boundary Value Analysis and Equivalence Partitioning for input validation
- Performance Testing for real-time image processing and notifications
- User Acceptance Testing (UAT) to validate usability and accessibility requirements

9. Item Pass/Fail Criteria

PASS:

- The feature performs according to specifications.
- No critical or blocking defects exist.
- System outputs are accurate and generated within acceptable response times.
- Notifications are correctly triggered.

FAIL:

- Application crashes occur.
- Incorrect outputs are produced.
- Features fail to satisfy requirements.
- Response times exceed acceptable limits.
- Notifications fail to operate correctly.

10. Suspension Criteria and Resumption Requirements

Suspension Criteria:

Testing activities may be suspended if:

- Critical defects prevent further testing.
- Backend services become unavailable.
- The application repeatedly crashes.
- AI modules fail to initialize correctly.

Resumption Requirements:

Testing activities will resume when:

- Critical defects have been resolved.
- Corrected functionality has been retested successfully.
- System stability has been verified.

11. Test Deliverables

- Software Test Plan Document
- Test Cases and Test Scripts
- Test Execution Reports
- Bug and Defect Logs
- Test Summary Report
- Performance Evaluation Report
- User Acceptance Testing Report
- Final Testing Documentation

12. Remaining Test Tasks

- Execute remaining test cases for low-light environments
- Validate moving obstacle detection scenarios

- Complete performance testing on multiple Android devices
- Perform regression testing after defect corrections
- Finalize User Acceptance Testing (UAT)
- Validate accessibility and usability requirements
- Conduct long-duration stability testing

13. Environmental Needs

Devices:

- Android smartphones (low-end, mid-range, and high-end)

Software:

- Android Studio
- FastAPI Server
- Python Environment
- Postman
- GitHub Repository

AI Models:

- YOLO11 Object Detection Model
- MiDaS Depth Estimation Model

Testing Tools:

- Android Emulator
- Logcat
- Performance Monitoring Tools
- Memory Profiler

Network:

- Stable Internet Connection for backend communication

14. Staffing and Training Needs

Personnel:

- QA Lead
- Two Manual Test Engineers
- Backend Tester
- Performance Tester
- Accessibility Tester

Training Requirements:

- Android application testing procedures
- AI-based application validation techniques
- Real-time system testing methodologies
- Accessibility testing procedures
- Defect reporting and management tools

15. Responsibilities

QA Lead:

Responsible for planning, coordinating, and monitoring all testing activities.

Test Engineers:

Responsible for preparing test cases, executing tests, logging defects, and retesting corrected functionality.

Developers:

Responsible for debugging, correcting defects, and supporting testing activities.

Project Manager:

Responsible for approving schedules, allocating resources, and monitoring project progress.

16. Schedule

Table 0-1-B :Testing schedule

Phase	Start Date	End Date
Test Planning	June 1, 2026	June 3, 2026
Test Case Development	June 4, 2026	June 7, 2026
Initial Test Execution	June 8, 2026	June 11, 2026
Bug Fixing and Regression Testing	June 12, 2026	June 15, 2026
Final Testing and User Acceptance Testing	June 16, 2026	June 18, 2026

17. Planning Risks and Contingencies

Risk: High latency during image processing.

Mitigation: Optimize image size and reduce unnecessary image processing operations.

Risk: Incorrect object detection.

Mitigation: Expand testing scenarios and validate results using additional datasets.

Risk: Device compatibility issues.

Mitigation: Test on multiple Android versions and device specifications.

Risk: Backend communication failure.

Mitigation: Implement timeout handling and reconnection mechanisms.

Risk: High memory consumption.

Mitigation: Optimize image processing frequency and improve memory management.

Risk: Notification failures.

Mitigation: Implement fallback notification mechanisms and extensive notification testing.

18. Approvals

QA Lead:

Name: _____

Signature: _____

Date: _____

Project Manager:

Name: _____

Signature: _____

Date: _____

Project Supervisor:

Name: _____

Signature: _____

Date: _____

Bug/Issue Log

Table 0-2-B: bug/issue log

Test Case ID	Feature	Description of Issue	Severity	Status	Notes
TC-003	Camera Module	Camera fails to initialize on certain devices	High	Open	Occurs on older Android versions
TC-007	Backend Communication	Image frames are not transmitted consistently	High	Investigating	Occurs under unstable network conditions
TC-012	YOLO11 Detection	Objects are occasionally missed under low-light conditions	High	Open	Requires model threshold adjustment
TC-016	MiDaS Depth Estimation	Inaccurate distance estimation for nearby obstacles	Medium	In Progress	Requires additional calibration
TC-020	Risk Assessment	Risk levels not updated correctly for moving objects	Medium	Open	Logic refinement required
TC-024	Text-to-Speech	Speech notifications occasionally fail to play	Medium	Fixed	Initialization issue resolved
TC-027	Vibration Feedback	Weak vibration intensity on some devices	Low	Fixed	Device-specific adjustments applied

Test Summary Report

Testing Period:

June 8, 2026 – June 18, 2026

Total Test Cases Executed: 50

- Passed: 42
- Failed: 8

Pass Rate: 84.0%

Fail Rate: 16.0%

Status Overview:

- Critical Bugs: 2
- High Severity Bugs: 3
- Medium Severity Bugs: 2
- Low Severity Bugs: 1

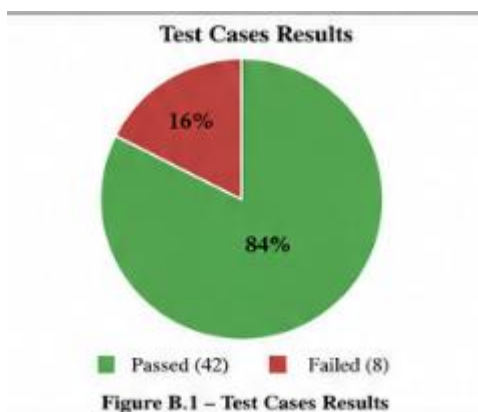


Figure 21: Test Cases Results

Professional Responsibility

Throughout the testing process, the project team adhered to software engineering standards and quality assurance practices to ensure that Smart Vision Assist provides reliable, safe, and accessible assistance for visually impaired users during indoor navigation.

Appendix C - Senior Project Summary Report

Project Title	Smart Vision Assist	
Supervisor(s)	Prof . Dr . Walid Rabie Dr . Yasmine Nagy	
Team members:	Names Aya Abdelhakam Khadija Mohamed Hana Khamis Habiba Aboraya	Registration Numbers 221010471 221001653 221010091 211002226
Project Deliverables	• Project Proposal: A detailed proposal outlining the project's objectives, scope, methodology, and expected outcomes. • Requirements Specification Document: Comprehensive documentation of the system requirements, including both functional and non-functional requirements. • System Design Document: Detailed documentation of the system architecture, component interactions, UML diagrams, and design decisions. • Implementation Plan: A step-by-step plan describing the development process, deployment strategy, and implementation activities. • Source Code: Complete source code of the Smart Vision Assist system, including the Android application, AI models integration, backend services, and user notification mechanisms. • Testing Reports: Documentation of unit testing, integration testing, system testing, performance testing, and user acceptance testing results. • User Documentation: User manuals, installation guides, and technical documentation for users, developers, and administrators. • Training Materials: Training documents and demonstration materials explaining how to use and maintain the Smart Vision Assist system.	

	• Final Report: A comprehensive report summarizing the project objectives, methodologies, implementation process, achievements, challenges, and future work. • Presentation: A presentation summarizing the project, its objectives, implementation, testing results, and contributions for supervisors and stakeholders.
Team Organization	The project team consists of interdisciplinary members, with each member contributing specialized skills and responsibilities: Project Manager • Role: Oversees the project, manages resources and timelines, and ensures alignment with project objectives. • Contribution: Coordinates team activities, monitors progress, facilitates communication, and manages project documentation. Lead Developer • Role: Leads the software development process and makes key technical decisions. • Contribution: Designs the system architecture, supervises implementation activities, and ensures adherence to technical standards. Artificial Intelligence Engineer • Role: Develops and integrates Artificial Intelligence models. • Contribution: Implements object detection and depth estimation models, optimizes model performance, and ensures real-time processing capabilities.

	Mobile Application Developer • Role: Develops the Android application. • Contribution: Designs and implements the user interface, integrates camera services, and develops speech and vibration feedback mechanisms. Backend Developer • Role: Develops backend services and communication interfaces. • Contribution: Implements API services, manages data processing, and ensures efficient communication between system components. Tester and Quality Assurance Specialist • Role: Performs testing and quality assurance activities. • Contribution: Develops test cases, executes testing procedures, identifies defects, and documents testing results. Documentation Specialist • Role: Prepares technical and user documentation. • Contribution: Produces reports, user manuals, installation guides, and project documentation. Each team member contributes from their area of expertise and collaborates closely with the rest of the team to ensure the successful development of Smart Vision Assist. Regular meetings and continuous communication are maintained throughout the project lifecycle to facilitate coordination and address challenges promptly.
--	--

Ethical Considerations	The Smart Vision Assist project addresses several ethical and safety considerations: • Data Privacy: Ensuring that camera data and user information are processed securely and are not stored or shared without authorization. • Accessibility: Providing an assistive solution that enhances independence and equal access to navigation support for visually impaired individuals. • Safety: Ensuring that warning notifications are accurate, timely, and reliable to avoid potential risks during navigation. • Transparency: Clearly communicating the capabilities and limitations of the system to users.
Social Impact	The Smart Vision Assist system is expected to have significant social impacts, including: • Enhanced Independence: Assisting visually impaired individuals in navigating indoor environments more safely and independently. • Improved Accessibility: Providing affordable assistive technology using widely available smartphone devices. • Increased Safety: Reducing navigation risks by providing real-time obstacle awareness and hazard notifications. • Technological Innovation: Encouraging the adoption of Artificial Intelligence and Computer Vision technologies for accessibility applications.
Professional Responsibility	Professional responsibility has been a fundamental aspect throughout the project:

	• Adherence to Standards: Following software engineering principles, development standards, and accessibility guidelines. • Accountability: Clearly defining roles and responsibilities for each project member. • Quality Assurance: Conducting comprehensive testing and validation to ensure system reliability, usability, and performance. • Continuous Improvement: Collecting feedback, evaluating system limitations, and identifying opportunities for future enhancements and improvements.
--	--

Supervisor Name	Signature
Prof . DR . Walid Rabie	
DR . Yasmine Nagy	